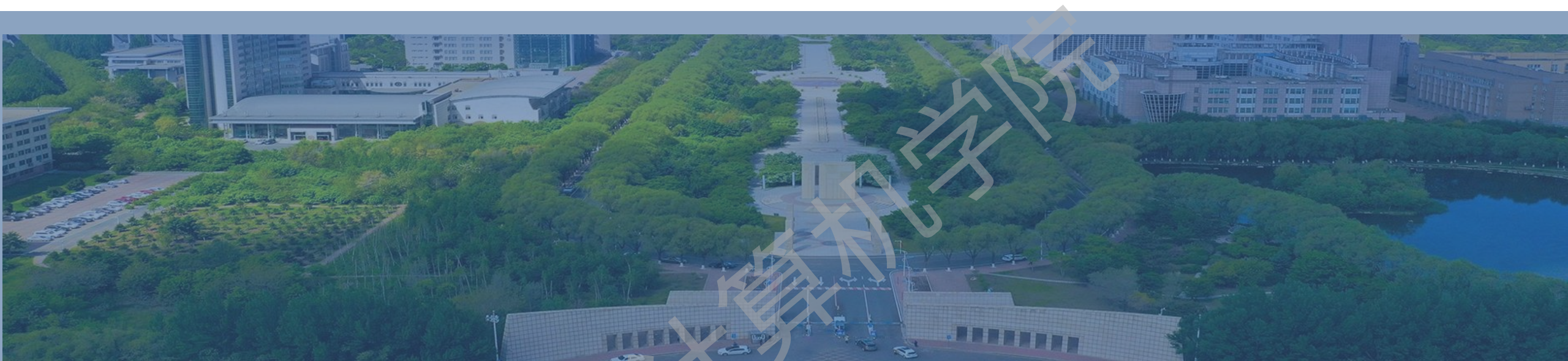




第六章 动态规划



目录

- 6.1 一般方法
- 6.2 多段图
- 6.3 货郎担问题
- 6.4 0/1背包问题
- 6.5 可靠性设计
- 6.6 最优二分检索树
- 6.7 矩阵链乘积问题
- 6.8 流水线调度问题
- 6.9 小结



6.1 一般方法

- 方法适用的问题特点
- 最优性原理
- 递推关系式设计
- 重叠子问题性质
- 动态规划的设计思想
- 动态规划的特点



方法适用的问题特点

- 有一类多阶段决策问题
 - 它可以被分成若干个阶段，每一个阶段都需要作出决策， i 阶段的决策仅依赖于 i 阶段当前的状态
 - 当每个阶段的决策都确定后，问题得到一个决策序列，进而计算出一个解
- 动态规划方法适用于解决求最优解的多阶段决策问题
 - 寻找最优解和最优决策序列

问题描述中一定存在目标函数

也称为多阶段决策优化问题，
或组合优化问题



动态规划的设计思想概要

20世纪50年代，贝尔曼等人根据多阶段决策问题的特性，提出了解决这类问题的“最优性原理”，从而创建了一种新的算法设计方法——**动态规划**

- 动态规划方法求解多阶段决策问题时：
 - 证明多阶段决策(最优)问题满足最优性原理
 - 设计递推关系式
 - 迭代实现程序



最优性原理(Principle of Optimality)



过程的最优决策序列具有如下性质： 无论过程的初始状态和初始决策是什么，其余的决策都必须相对于初始决策所产生的状态构成一个最优决策序列。

$$S_0 \xrightarrow{x_1} S_1 \xrightarrow{x_2} S_2 \xrightarrow{x_3} \dots \xrightarrow{x_n} S_n$$

如果原问题包含子问题，那么原问题的部分最优决策序列一定是其子问题的最优决策序列

- 问题满足最优性原理，则称其具有**最优子结构性**质
- 最优性原理证明思想
 - ① 确定最优决策序列的形式
 - ② 确定初始状态和初始决策
 - ③ 确定初始决策所产生的状态
 - ④ 证明其余决策相对于③是最优决策序列

思考1：任意子问题的最优决策序列都是原问题的部分最优决策序列么？

思考2：最优性原理的逆否命题是什么？

以0/1背包问题为例

- 问题描述：对于 n 个物品，容量为 M 的背包，要求物品或者整件装入背包中，或者根本不装入，即 x_i 限定只能取0或1值。

$$\begin{aligned} &\text{极大化} \quad \sum_{1 \leq i \leq n} p_i x_i \\ &\text{约束条件} \quad \sum_{1 \leq i \leq n} w_i x_i \leq M \\ &x_i = 0 \text{ 或 } 1, 1 \leq i \leq n \end{aligned}$$

英文字母 L

- 用 $\text{KNAP}(l, j, X)$ 来表示这个问题，函数值表示最大效益和。对于 n 个物品，容量为 M 的背包，就可表示为 $\text{KNAP}(1, n, M)$

问题最优解

问题边界

约束条件

0/1背包问题的最优性原理证明

- 设 $x_1, x_2, \dots, x_n$ 是最优序列。
- 若 $x_1=0$ ，则 $x_2, x_3, \dots, x_n$ 必须相对于 $\text{KNAP}(2, n, M)$ 问题构成一个最优序列。如若不然，则 $x_1, x_2, \dots, x_n$ 就不是 $\text{KNAP}(1, n, M)$ 的最优序列。
- 若 $x_1=1$ ，则 $x_2, x_3, \dots, x_n$ 必须是 $\text{KNAP}(2, n, M-w_1)$ 的最优序列。如若不然，则必有另一个0/1序列 $z_2, z_3, \dots, z_n$ 使得 $\sum w_i z_i \leq M-w_1 (2 \leq i \leq n)$ 且 $\sum p_i z_i > \sum p_i x_i (2 \leq i \leq n)$ 。因此，序列 $x_1, z_2, z_3, \dots, z_n$ 是一个对问题 $\text{KNAP}(1, n, M)$ 具有更大效益值的序列。这与假设相矛盾。
- 所以0/1背包问题的最优性原理成立。

思考：以0/1背包为例，分析为什么最优性原理能够帮助提高算法效率？

递推关系式设计

- 向前处理法(forward approach)
 - 从最后阶段开始,逐步向前递推,即根据 $x_{i+1}, \dots, x_n$ 的那些最优决策序列来列出求取 x_i 决策值的关系式。
- 向后处理法(backward approach)
 - 从初始阶段开始,逐步向后递推,即根据 $x_1, \dots, x_{i-1}$ 的那些最优决策序列来列出求取 x_i 决策值的关系式。



0/1背包问题向前处理法的递推关系式

- 根据最优性原理：
 - $\text{KNAP}(1, n, M) = \max\{\text{KNAP}(2, n, M), \text{KNAP}(2, n, M - w_1) + p_1\}$
 - 参数 n 不起作用，舍弃后简化函数
- 设 $g_j(X) = \text{KNAP}(j+1, n, X)$ ，则 $g_0(M) = \text{KNAP}(1, n, M)$
 - 上式简化为： $g_0(M) = \max\{g_1(M), g_1(M - w_1) + p_1\}$
- 推知一般： $g_j(X) = \max\{g_{j+1}(X), g_{j+1}(X - w_{j+1}) + p_{j+1}\}$

$$g_n(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$



0/1背包问题向后处理法的递推关系式

- 设 $f_i(X) = \text{KNAP}(1, i, X)$, 则 $f_n(M) = \text{KANP}(1, n, M)$
- 那么 $f_n(M)$ 的递归关系式可表示为: $f_n(M) = \max \{ f_{n-1}(M), f_{n-1}(M - w_n) + p_n \}$
- 对于任意的 $f_i(X)$, $i > 0$ 有以下公式:

$$f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X - w_i) + p_i \}$$

$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

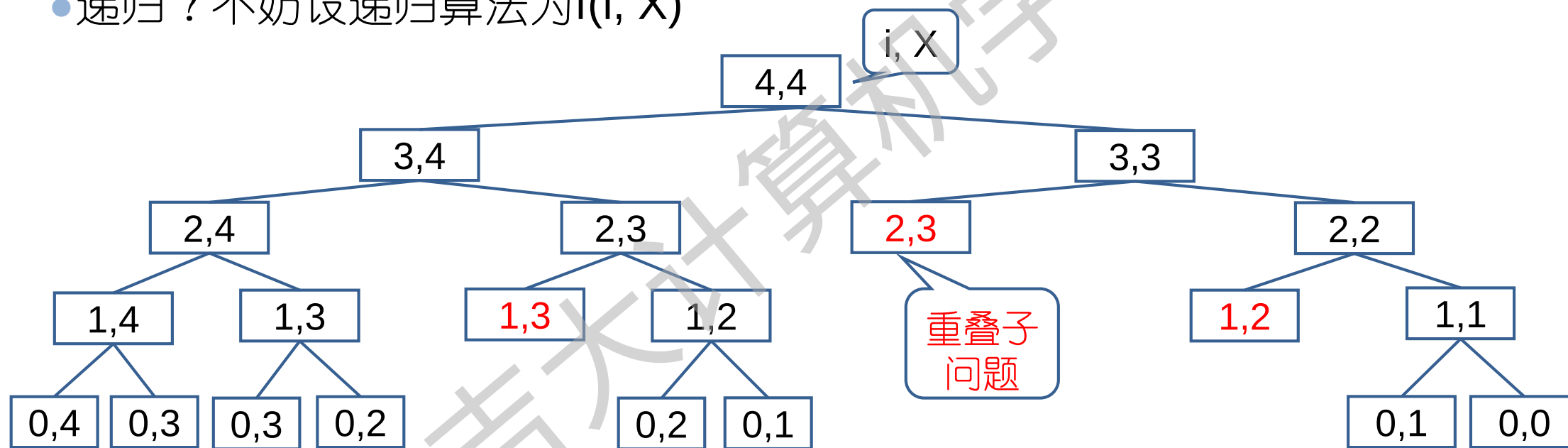
思考：如何设计算法？根据递推关系式如此顺畅的自顶向下递归求解，为什么还要自底向上迭代求解？



重叠子问题性质



- 向后处理递推关系式: $f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X-w_i)+p_i \}$
- 例: $n=4, (p_1, p_2, p_3, p_4)=(1, 2, 5, 6), (w_1, w_2, w_3, w_4)=(1, 1, 1, 1), M=4$
 - 递归? 不妨设递归算法为 $f(i, X)$



- **重叠子问题性质**: 求解一个问题时, 同一个子问题被多次重用。
- 自底向上迭代求解效率更高。重叠子问题越多, 效率越高

动态规划的设计思想

- 动态规划方法求解问题时：

- 证明多阶段决策(最优)问题满足最优性原理

- 设计递推关系式

刻画问题规模

- 设计函数表达形式：函数值代表问题最优解；参数集代表问题边界和约束条件

- 根据最优性原理确定原问题和子问题的关系

- 自底向上方式设计程序

- 从最小的子问题开始迭代计算，直到达到原始问题规模

- 计算过程保存所有子问题的最优解和最优决策

备忘录

标记函数

针对一些问题，动态规划法比蛮力法(穷举)在时间复杂度上得到了实质的改进

再谈动态规划的迭代实现

- 自底向上计算子问题：
 - 每个子问题只计算一次。先计算出所有规模为1的子问题，再计算所有规模为2的子问题，……，直到规模 n 为止。
- 定义“**备忘录**” (和标记函数)记录子问题的最优解和最优决策：
 - 即定义一个存储空间，通常是表格形式，保存每个子问题的最优解(和最优决策)，记录最优解的表格称为“备忘录”。
 - 重用子问题时，直接查询备忘录。



动态规划的特点

- 特点
 - 最优子结构性质(满足最优性原理)
 - 重叠子问题性质
- 优点
 - 设计思想优于蛮力法，实现方法优于递归法
- 缺点
 - 如果子问题重叠程度低，动态规划的时间复杂度改进不明显
 - 过大的空间需求成为动态规划算法效率低下的主要因素





6.2 多段图问题

- 问题描述
- 最优性原理证明
- 多段图向前处理递推关系式
- 向前处理的计算过程
- 向前处理算法6.1及执行过程
- 多段图向后处理递推关系式
- 向后处理的计算过程
- 多段图的应用

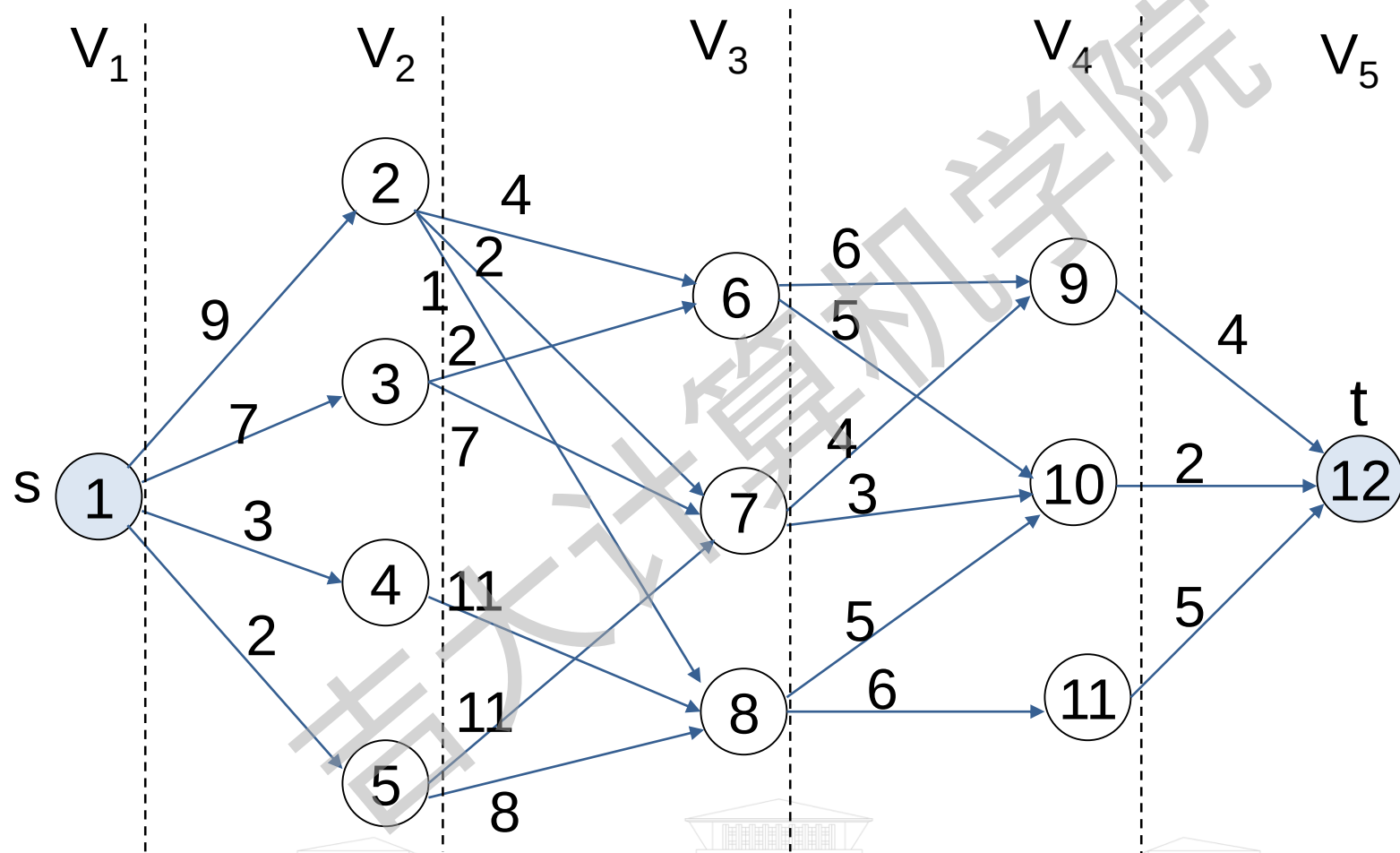


问题描述

- 多段图 $G=(V,E)$ 是一个有向图。它具有如下特性：
 - 图中的结点被划分成 $k \geq 2$ 个不相交的集合 V_i , $1 \leq i \leq k$, 其中 V_1 和 V_k 分别只有一个结点 s (源点) 和 t (汇点)
 - 图中所有的边 $\langle u, v \rangle$ 均具有如下性质: 若 $u \in V_i$, 则 $v \in V_{i+1}$, $1 \leq i \leq k$, 且每条边 $\langle u, v \rangle$ 均附有成本 $c(u, v)$
- 从 s 到 t 的一条路径成本是这条路径上边的成本和。
- 多段图问题是求由 s 到 t 的 **最小成本** 路径。



多段图示例



多段图问题的最优性原理证明

- 假设 $s, v_2, v_3, \dots, v_{k-1}, t$ 是一条由 s 到 t 的最短路径。
- 再假设从源点 s 开始，已作出了到结点 v_2 的决策， v_2 就是初始决策所产生的状态。
- 如果把 v_2 看成是原问题的一个子问题的初始状态，解决这个子问题就是找出一条由 v_2 到 t 的最短路径。
- 这条最短路径显然是 $v_2, v_3, \dots, v_{k-1}, t$ 。如果不然，设 $v_2, q_3, \dots, q_{k-1}, t$ 由 v_2 到 t 的更短路径，则 $s, v_2, q_3, \dots, q_{k-1}, t$ 肯定比 $s, v_2, v_3, \dots, v_{k-1}, t$ 路径短，这与假设矛盾。
- 因此最优性原理成立。



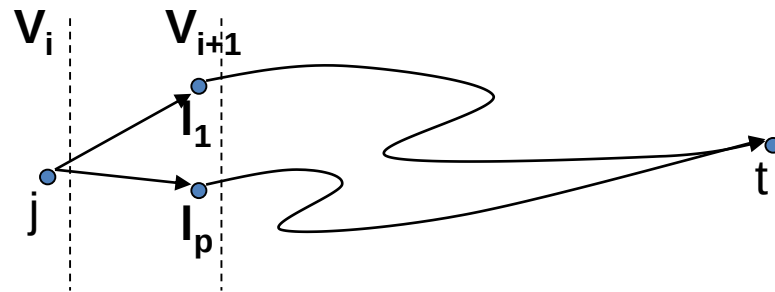
多段图向前处理递推关系式

- 设 $P(i, j)$ 是一条从 V_i 中的结点 j 到汇点 t 的最小成本路径, $COST(i, j)$ 表示这条路径的成本, 根据向前处理方法:

$$COST(i, j) = \min\{c(j, l) + COST(i+1, l)\}$$

其中: $l \in V_{i+1}$, $\langle j, l \rangle \in E$, $c(j, l)$ 表示该边的成本。

- 对于 k 段图, 当 $i=k-1$ 时(初值)
 - 如果 $\langle j, t \rangle \in E$, $COST(k-1, j) = c(j, t)$
 - 否则, $COST(k-1, j) = \infty$



最优解

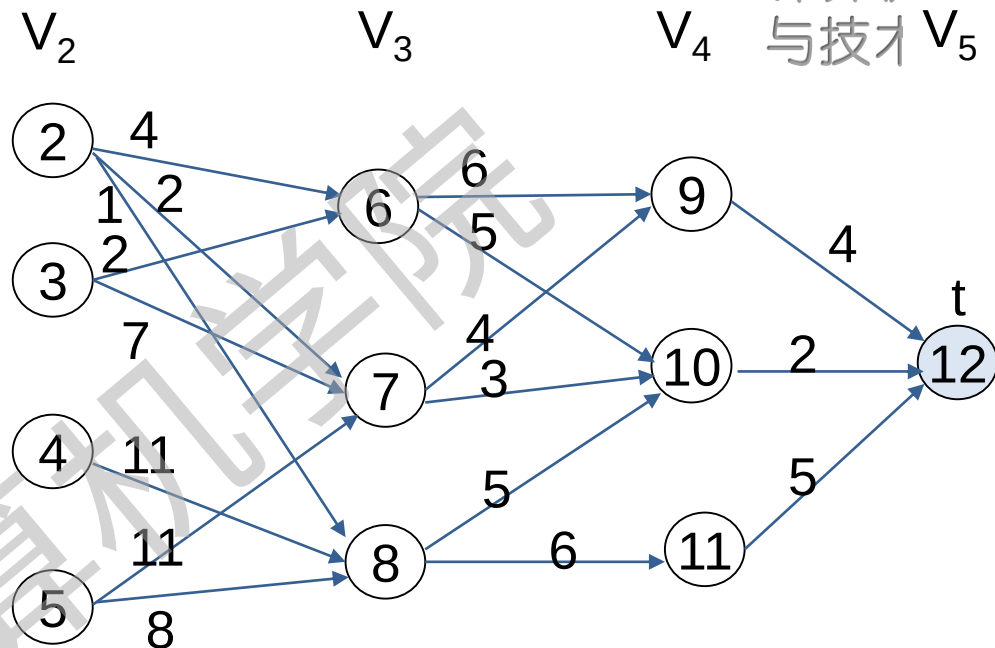
$$\text{COST}(i, j) = \min\{c(j, l) + \text{COST}(i+1, l)\}$$

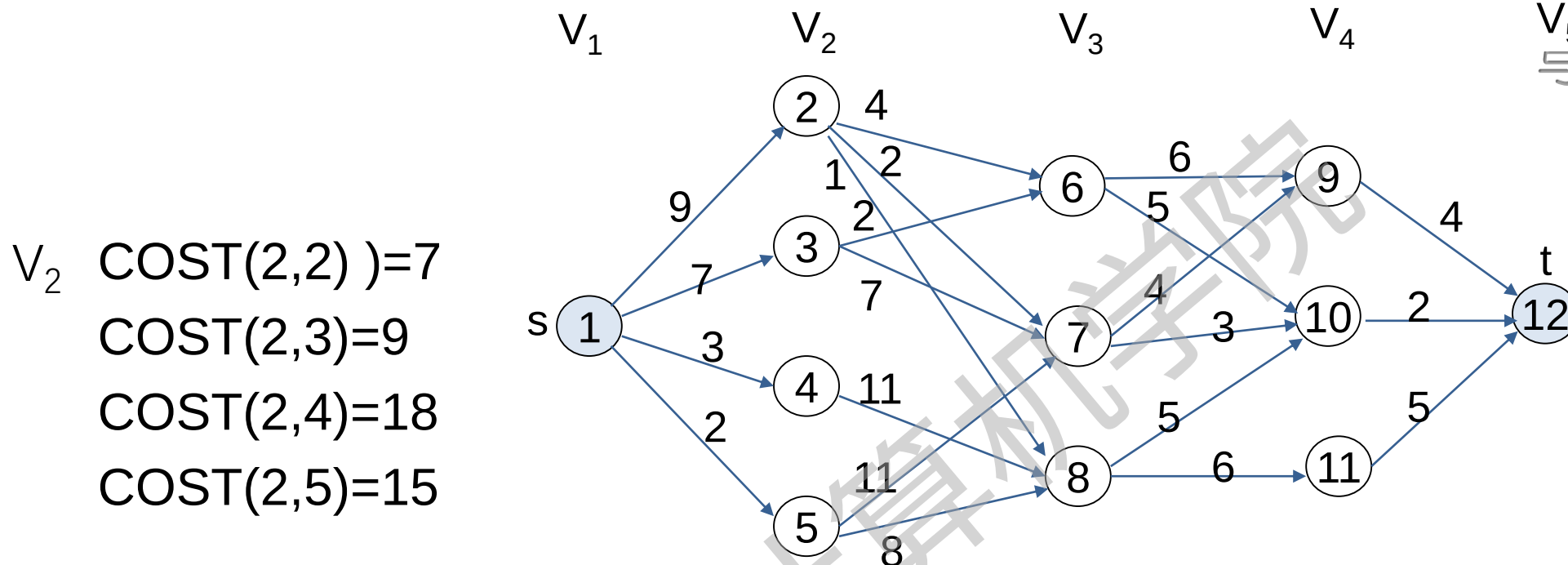
从最小的子问题
开始迭代计算

$$\begin{aligned} V_4 \quad & \text{COST}(4, 9) = c(9, 12) = 4 \\ & \text{COST}(4, 10) = c(10, 12) = 2 \\ & \text{COST}(4, 11) = c(11, 12) = 5 \end{aligned}$$

$$\begin{aligned} V_3 \quad & \text{COST}(3, 6) = \min\{c(6, 9) + \text{COST}(4, 9), c(6, 10) + \text{COST}(4, 10)\} = \min\{6 + 4, 5 + 2\} = 7 \\ & \text{COST}(3, 7) = \min\{c(7, 9) + \text{COST}(4, 9), c(7, 10) + \text{COST}(4, 10)\} = \min\{4 + 4, 3 + 2\} = 5 \\ & \text{COST}(3, 8) = \min\{c(8, 10) + \text{COST}(4, 10), c(8, 11) + \text{COST}(4, 11)\} = \min\{5 + 2, 6 + 5\} = 7 \end{aligned}$$

$$\begin{aligned} V_2 \quad & \text{COST}(2, 2) \\ & = \min\{c(2, 6) + \text{COST}(3, 6), c(2, 7) + \text{COST}(3, 7), c(2, 8) + \text{COST}(3, 8)\} = \min\{4 + 7, 2 + 5, 1 + 7\} \\ & = 7 \end{aligned}$$





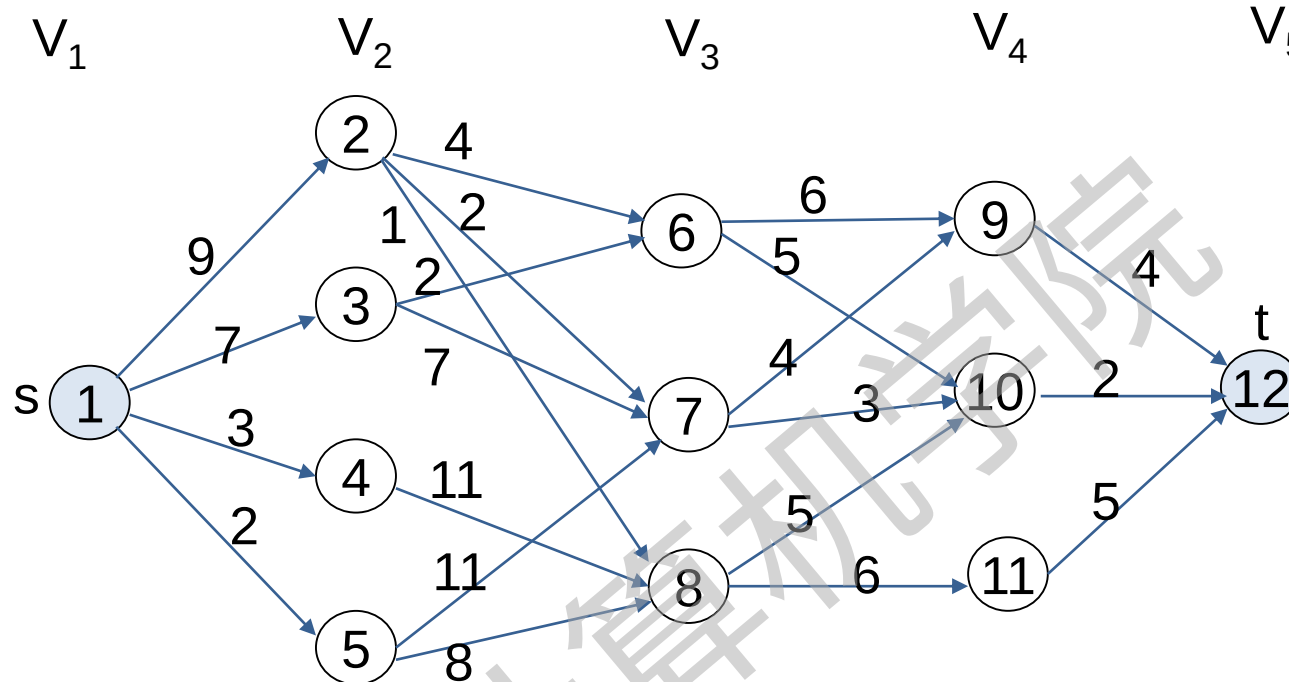
$$\begin{aligned}
 V_2 \quad & \text{COST}(2,2) = 7 \\
 & \text{COST}(2,3) = 9 \\
 & \text{COST}(2,4) = 18 \\
 & \text{COST}(2,5) = 15
 \end{aligned}$$

$$\begin{aligned}
 V_1 \quad & \text{COST}(1,1) = \min\{c(1,2) + \text{COST}(2,2), c(1,3) + \text{COST}(2,3), \\
 & \quad c(1,4) + \text{COST}(2,4), c(1,5) + \text{COST}(2,5)\} \\
 & = \min\{9 + 7, 7 + 9, 3 + 18, 2 + 15\} = 16
 \end{aligned}$$

思考：为什么优于蛮力法(穷举)？



备忘录

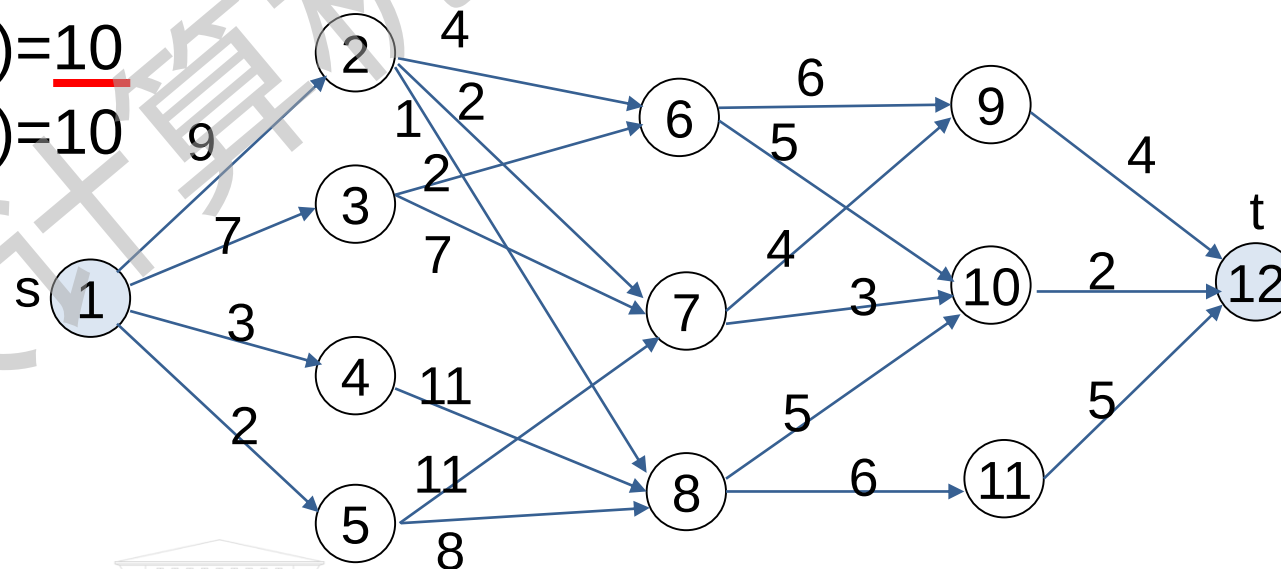


$COST(i,j)$	1	2	3	4	5	6	7	8	9	10	11	12
$i=4$	∞	∞	∞	∞	∞	∞	∞	∞	4	2	5	
$i=3$	-	-	-	-	-	7	5	7	-	-	-	
$i=2$	-	7	9	18	5	-	-	-	-	-	-	
$i=1$	16	-	-	-	-	-	-	-	-	-	-	

最优决策序列

- 在计算每个 $\text{COST}(i, j)$ 的同时, 记下每个状态 (结点 j) 所做出的决策 l , 令 $D(i, j) = l$, 则容易求出这条最小成本的路径

V_1	V_2	V_3
$D(1, 1) = \underline{2}$	$D(2, 2) = \underline{7}$	$D(3, 6) = 10$
	$D(2, 3) = 6$	$D(3, 7) = \underline{10}$
	$D(2, 4) = 8$	$D(3, 8) = 10$
	$D(2, 5) = 8$	



最小成本的路径为: **1 → 2 → 7 → 10 → 12**

标记函数

D(i,j)	1	2	3	4	5	6	7	8	9	10	11	12
i=4									12	12	12	
i=3						10	10	10				
i=2		7	6	8	8							
i=1	2											

思考：是否可以减小存储空间？

对结点进行编号，从 V_1 开始 s 编为1号，然后 V_2 中的结点依次编为2,3,4,5号，按此规则编下去，可以将COST, D中表示段数的第一个下标 i 省略掉，改为一维数组存储。

算法6.1 多段图的向前处理算法

procedure FGRAPH(E,k,n,P) P(1:k)保存最小成本路径上结点

real COST(n), int D(n-1), P(k), r, j, k, n

COST(n) $\leftarrow$ 0

for j $\leftarrow$ n-1 to 1 by -1 do

寻找结点r, 满足 $\langle j, r \rangle \in E$ 且使 $c(j,r)+COST(r)$ 值最小

COST(j) $\leftarrow$ c(j,r)+COST(r);

D(j) $\leftarrow$ r;

repeat

P(1) $\leftarrow$ 1; P(k) $\leftarrow$ n

for j $\leftarrow$ 2 to k-1 do P(j) $\leftarrow$ D(P(j-1)) repeat

end FGRAPH

- 时间复杂度: $\Theta(n+e)$
 - n是图G中结点个数。
 - e是图G中边的个数。
- 空间复杂度: $\Theta(n+e)$
 - 邻接表的存储空间
 - 除E外, 还需要COST、D、P

多段图可用邻接表存储



FGRAPH算法执行过程

$COST(1)=16$

$D(1)=2$

$COST(5)=15$

$COST(4)=18$

$COST(3)=9$

$COST(2)=7$

$D(5)=8$

$D(4)=8$

$D(3)=6$

$D(2)=7$

$COST(8)=7$

$COST(7)=5$

$COST(6)=7$

$D(8)=10$

$D(7)=10$

$D(6)=10$

$COST(11)=5$

$COST(10)=2$

$COST(9)=4$

$D(11)=12$

$D(10)=12$

$D(9)=12$

	1	2	3	4	5	6	7	8	9	10	11	12
COST	16	7	9	18	15	7	5	7	4	2	5	0
	1	2	3	4	5	6	7	8	9	10	11	
D	2	7	6	8	8	10	10	10	12	12	12	
	1	2	3	4	5							
P	1	2	7	10	12							

保存各阶段子问题的最优决策和最优解：

COST: 备忘录，存储解；

D: 标记函数，存储决策

多段图向后处理递推关系式

- 设 $BP(i, j)$ 是一条从源点 s 到 V_i 中的结点 j 的最小成本路径, $BCOST(i, j)$ 表示这条路径的成本, 根据向后处理方法:

$$BCOST(i, j) = \min\{BCOST(i-1, l) + c(l, j)\}$$

其中: $l \in V_{i-1}$, $\langle l, j \rangle \in E$, $c(l, j)$ 表示该边的成本。

- 对于 k 段图, 当 $i=2$ 时(初值)
 - 如果 $\langle s, j \rangle \in E$, $BCOST(2, j) = c(s, j)$
 - 否则, $BCOST(2, j) = \infty$



最优解

$$\text{BCOST}(i, j) = \min\{\text{BCOST}(i-1, l) + c(l, j)\}$$

简化为:

$$\text{BCOST}(j) = \min\{\text{BCOST}(l) + c(l, j)\}$$

$$\text{BCOST}(2)=9$$

$$\text{BCOST}(3)=7$$

$$\text{BCOST}(4)=3$$

$$\text{BCOST}(5)=2$$

$$D(2)=1$$

$$D(3)=\underline{1}$$

$$D(4)=1$$

$$D(5)=1$$

$$\text{BCOST}(6)=9$$

$$\text{BCOST}(7)=11$$

$$\text{BCOST}(8)=10$$

$$D(6)=\underline{3}$$

$$D(7)=2$$

$$D(8)=2$$

$$\text{BCOST}(9)=15$$

$$\text{BCOST}(10)=14$$

$$\text{BCOST}(11)=16$$

$$D(9)=6$$

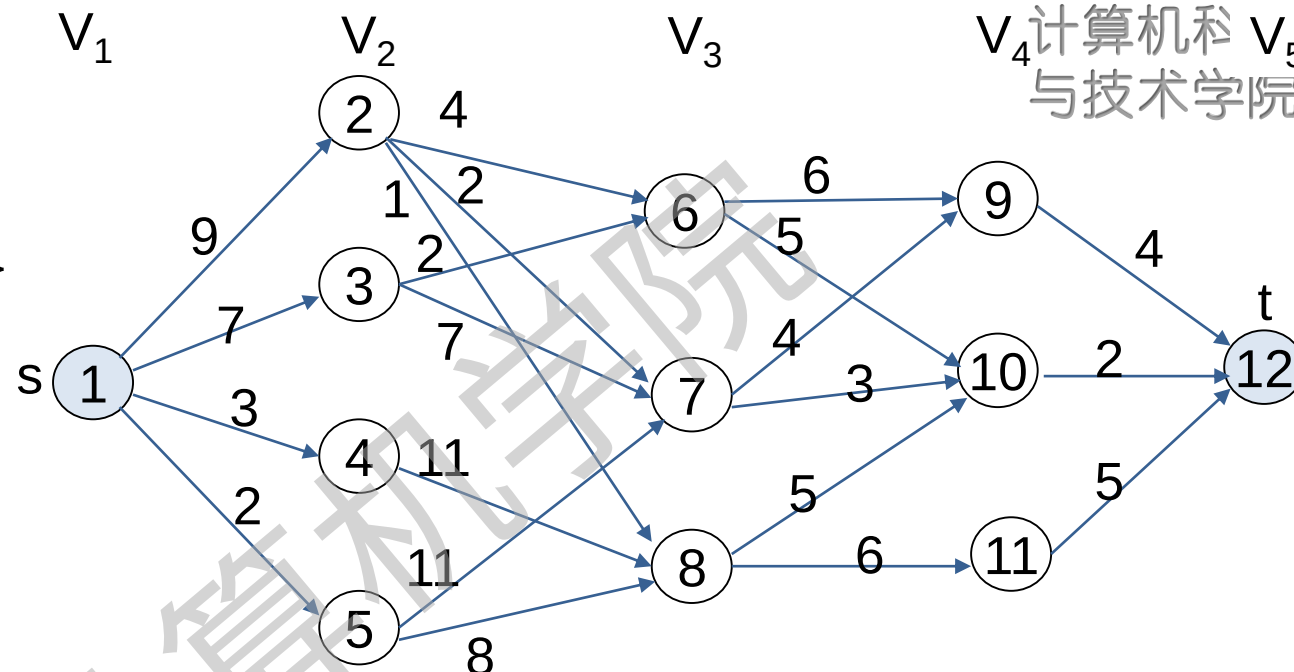
$$D(10)=\underline{6}$$

$$D(11)=8$$

$$\text{BCOST}(12)=16$$

$$D(12)=\underline{10}$$

最小成本的路径为: $1 \rightarrow 3 \rightarrow 6 \rightarrow 10 \rightarrow 12$



计算机科学与技术学院

思考: 如何设计多段图向后处理算法? 向后处理算法中如何存储多段图?

多段图的应用

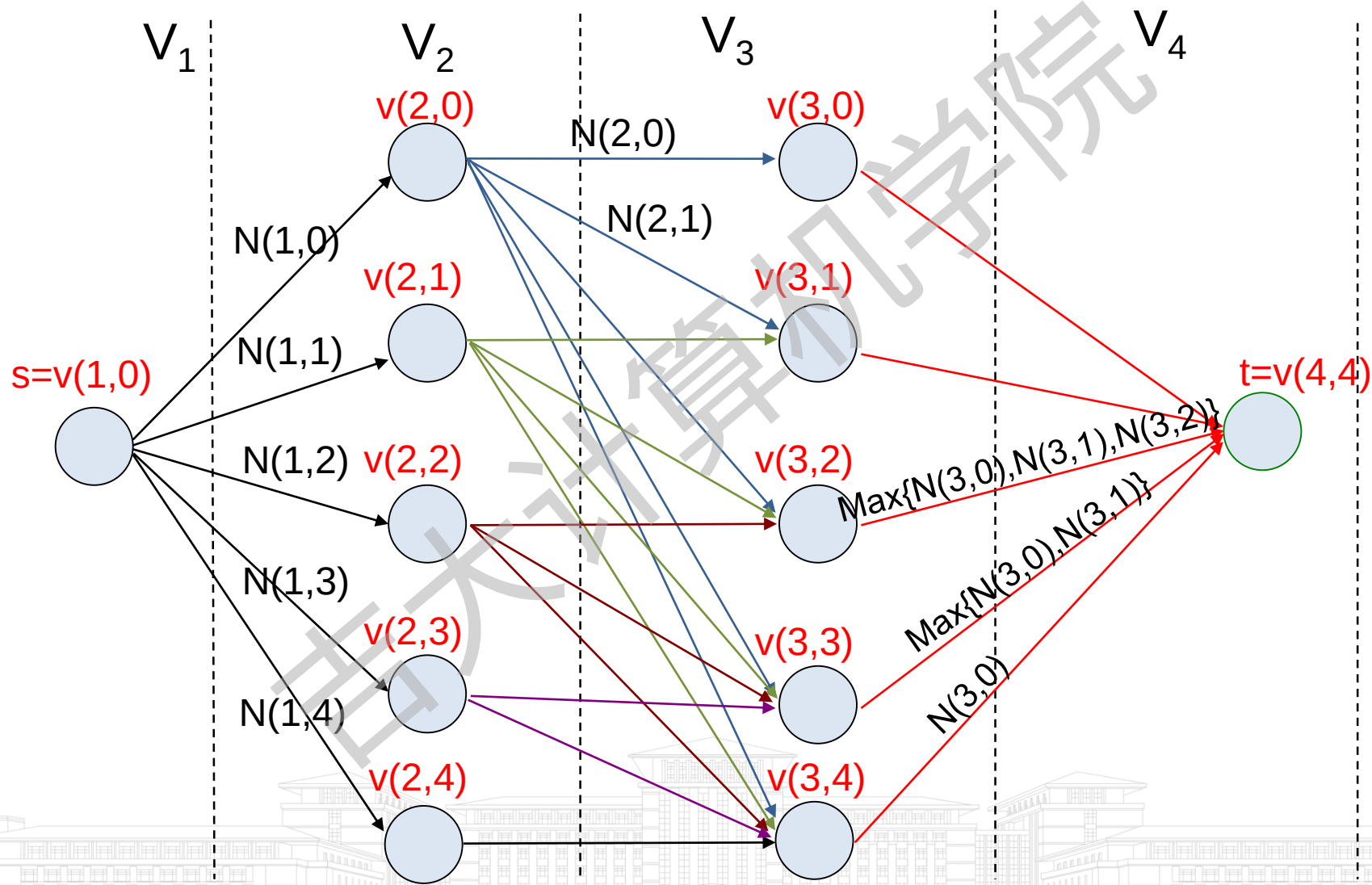
- 考虑把 n 个资源分配给 r 个项目的问题，要求使得总净利达到最大值。
 - $N(i, j)$ 表示把 j 个资源分配给项目 i 所获得的利润， $0 \leq j \leq n$ ， $1 \leq i \leq r$
- 用多段图来示该问题

$r+1$ 段图

 - $V(i, j)$ 表示一共把 j 个资源分配给了项目 $1, 2, \dots, i-1$ ，即前 $i-1$ 个项目。
 - $\langle V(i, j), V(i+1, l) \rangle$ 表示项目 i 分配到 $l-j$ 个资源，则获取到 $N(i, l-j)$ 的利润。从 $V(1, 0)$ 到 $V(i+1, l)$ 的路径表示前 i 个项目一共分配到 l 个资源。
 - 当 $j \leq n$ 且 $i=r$ 时，边具有形式 $\langle V(r, j), V(r+1, n) \rangle$ ，成本值为 $\max_{0 \leq p \leq n-j} \{N(r, p)\}$ 。



4个资源分配给3个项目的资源分配问题



6.3 货郎担问题

- 问题描述
- 最优性原理证明
- 递推关系式
- 计算过程
- 时间复杂度分析



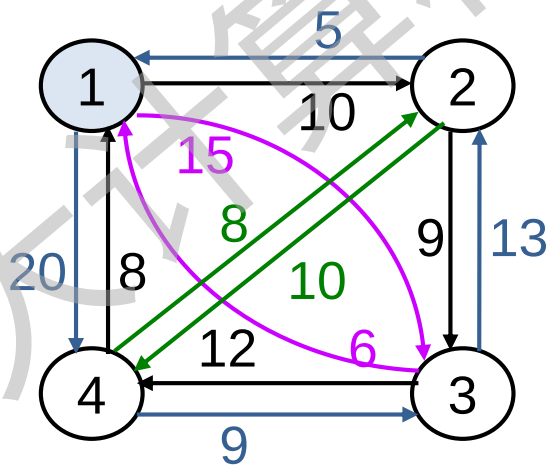
问题描述

- 货郎担问题：某售货员要到若干个村庄售货，各村庄之间的路程是已知的，为了提高效率，售货员决定从所在商店出发，到每个村庄售货一次，最后返回商店，问他应选择一条什么路线才能使所走的总路程最短。
- 类似问题
 - 邮车收集邮件的邮路问题
 - 机械手臂在装配线上的移动问题
 - 产品的生产安排问题



问题形式化表示

- 设 $G(V, E)$ 是一个具有边成本 c_{ij} 的有向图, G 的一条周游路线是包含 V 中每个结点的一个有向环, 周游路线的成本是此路线上所有边的成本之和。
- 用结点序列表示路线, 求最小成本的周游路线。



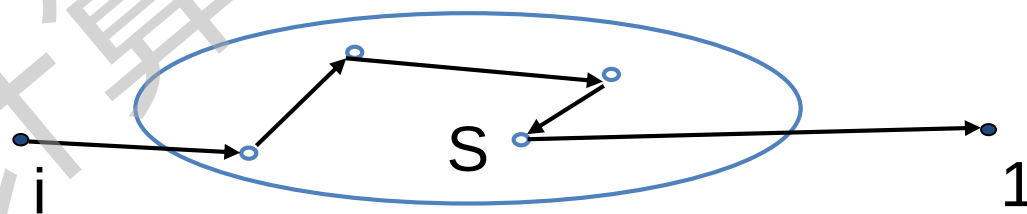
最优性原理证明

- 假设周游路线是开始于结点1并终止于结点1的一条简单路径。
 - 每一条周游路线都是由一条边 $\langle 1, k \rangle$ 和一条由结点k到结点1的简单路径所组成的，其中 $k \in V - \{1\}$ 。
 - 这条由结点k到结点1的路径通过 $V - \{1, k\}$ 的每个结点各一次。
- 如果这条周游路线是最优的，则这条由k到1的路径必定是通过 $V - \{1, k\}$ 中所有结点的由k到1的最短路径。
- 因此最优性原理成立。

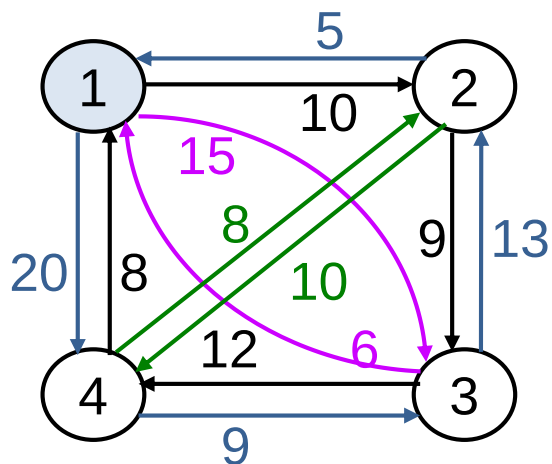


递推关系式

- 设 $g(i, S)$ 是由结点 i 开始，通过 S 中的所有结点，在结点 1 终止的一条最短路径长度。
 - $g(1, V-\{1\})$ 是一条最优的周游路线长度
 - 于是可以得到： $g(1, V-\{1\}) = \min_{2 \leq k \leq n} \{c_{1k} + g(k, V-\{1, k\})\}$
- 递推公式 $g(i, S) = \min_{j \in S} \{c_{ij} + g(j, S-\{j\})\}$
 初始值 $g(i, \emptyset) = c_{i1}, 1 < i \leq n$
- 求解过程
 - 求 $|S|=k$ 的所有 $g(i, S)$, $k=0, 1, \dots, n-1$;
 - 当 $k < n-1$ 时, $i \neq 1, 1 \notin S$ 且 $i \notin S$
 - 当 $k = n-1$ 时, 求 $g(1, V-\{1\})$, 问题得解。



计算过程



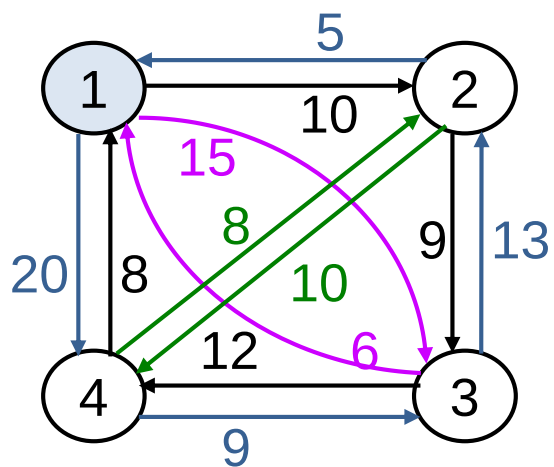
	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

$|S|=0$

- $g(2, \emptyset) = c_{21} = 5$
- $g(3, \emptyset) = c_{31} = 6$
- $g(4, \emptyset) = c_{41} = 8$

$|S|=1$

- $g(2, \{3\}) = c_{23} + g(3, \emptyset) = 9 + 6 = 15$
- $g(2, \{4\}) = c_{24} + g(4, \emptyset) = 10 + 8 = 18$
- $g(3, \{2\}) = c_{32} + g(2, \emptyset) = 13 + 5 = 18$
- $g(3, \{4\}) = c_{34} + g(4, \emptyset) = 12 + 8 = 20$
- $g(4, \{2\}) = c_{42} + g(2, \emptyset) = 8 + 5 = 13$
- $g(4, \{3\}) = c_{43} + g(3, \emptyset) = 9 + 6 = 15$



	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

- $|S|=2$
- $g(2, \{3, 4\}) = \min\{c_{23} + g(3, \{4\}), c_{24} + g(4, \{3\})\}$
 $= \min\{9 + 20, 10 + 15\} = 25$
 - $g(3, \{2, 4\}) = \min\{c_{32} + g(2, \{4\}), c_{34} + g(4, \{2\})\}$
 $= \min\{13 + 18, 12 + 13\} = 25$
 - $g(4, \{2, 3\}) = \min\{c_{42} + g(2, \{3\}), c_{43} + g(3, \{2\})\}$
 $= \min\{8 + 15, 9 + 18\} = 23$

- $|S|=3$
- $g(1, \{2, 3, 4\}) = \min\{c_{12} + g(2, \{3, 4\}), c_{13} + g(3, \{2, 4\}), c_{14} + g(4, \{2, 3\})\}$
 $= \min\{10 + 25, 15 + 25, 20 + 23\} = 35$

可得这条最优周游路线是: $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$

思考1: 从2出发的最优周游路线是那一条?

思考2: 和贪心法相比为什么能够保证获得最优解?

备忘录和标记函数

	$g(i,S)$	1	2	3	4
$k=0$	$\emptyset$	-	5	6	8
$k=1$	$\{2\}$	-	-	18	13
	$\{3\}$	-	15	-	15
	$\{4\}$	-	18	20	-
$k=2$	$\{2,3\}$	-	-	-	23
	$\{2,4\}$	-	-	25	-
	$\{3,4\}$	-	25	-	-
$k=3$	$\{2,3,4\}$	35	-	-	-

	$D(i,S)$	1	2	3	4
$\emptyset$	-	1	1	1	1
$\{2\}$	-	-	2	2	2
$\{3\}$	-	3	-	3	3
$\{4\}$	-	4	4	-	-
$\{2,3\}$	-	-	-	2	2
$\{2,4\}$	-	-	4	-	-
$\{3,4\}$	-	4	-	-	-
$\{2,3,4\}$	2	-	-	-	-

链表式存储时，
会去掉冗余

最优周游路线: $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$

时间复杂度分析

$$g(i, S) = \min_{j \in S} \{c_{ij} + g(j, S - \{j\})\}$$

- $|S|=k$, $k=0, 1, \dots, n-2$ 时, 对于 k 的每一种取值, $g(i, S)$ 中的结点 i 都有 $n-1$ 种选择, 即 $i=2, 3, \dots, n$, 对应每个 i 的 S 的个数为 C_{n-2}^k , S 是 $V - \{1, i\}$ 的子集。

k 的每一种取值对应的函数求解次数为: $(n-1)C_{n-2}^k$

k 的所有取值对应的函数求解次数为: $\sum_{k=0}^{n-2} (n-1)C_{n-2}^k = (n-1)2^{n-2}$

又知求函数 $g(i, S)$ 的值要进行 $|S|-1$ 比较: $\sum_{k=0}^{n-2} (n-1)(k-1)C_{n-2}^k = \Theta(n^2 2^n)$

优于枚举法的结果 $O(n!)$,
但随着 n 的增大, 该方法亦不可取。

6.4 0/1背包问题

- 问题回顾
- 计算过程
- 迭代算法与算法变型
- 递推关系式图解
- 序偶对法分析
- 序偶对法计算过程
- 序偶对法的复杂度
- 序偶对法的改进

算法变型略



问题回顾

- 0/1背包问题要求物品或者整件装入背包中, 或者根本不装入(即不能装入物品的一部分), 所以 x_i 限定只能取0或1值。

$$\begin{aligned} &\text{极大化} \quad \sum_{1 \leq i \leq n} p_i x_i && x_i = 0 \text{ 或 } 1, \quad p_i > 0 \\ &\text{约束条件} \quad \sum_{1 \leq i \leq n} w_i x_i \leq M && w_i > 0, \quad 1 \leq i \leq n \end{aligned}$$

- 最优性原理证明(略)
- 递推关系式: $f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X-w_i)+p_i \}$

$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

思考: 0/n背包(多重背包)问题是指 x_i 取值范围是自然数, 如何形式化描述问题和设计递推关系式?

计算过程

$$f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X-w_i)+p_i \}$$

$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

- 实例： $n=3, (w_1, w_2, w_3)=(2, 3, 4), (p_1, p_2, p_3)=(1, 2, 5), M=6$
- 备忘录 $F(i, X)$

$F(i, X)$	$X=0$	1	2	3	4	5	$6=M$
0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1
2							
3							

初值

$$f_1(X) = \begin{cases} -\infty & X < 0 \\ \max\{0, -\infty + 1\} = 0 & 0 \leq X < 2 \\ \max\{0, 0 + 1\} = 1 & 2 \leq X \leq 6 \end{cases}$$



递归关系式计算过程

$$f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X-w_i)+p_i \}$$

$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

- 实例： $n=3, (w_1, w_2, w_3)=(2, 3, 4), (p_1, p_2, p_3)=(1, 2, 5), M=6$
- 备忘录 $F(i, X)$

$F(i, X)$	0	1	2	3	4	5	6=M
$i=0$	0	0	0	0	0	0	0
$i=1$	0	0	1	1	1	1	1
$i=2$	0	0	1	2	2	3	3
$i=3$							

$$f_2(X) = \begin{cases} -\infty & X < 0 \\ 0 & 0 \leq X < 2 \\ 1 & 2 \leq X < 3 \\ \max\{1, 0+2\}=2 & 3 \leq X < 5 \\ \max\{1, 1+2\}=3 & 5 \leq X \leq 6 \end{cases}$$



递归关系式计算过程

$$f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X-w_i)+p_i \}$$

$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

- 实例： $n=3, (w_1,w_2,w_3)=(2,3,4), (p_1,p_2,p_3)=(1,2,5), M=6$
- 备忘录 $F(i,X)$

$F(i,X)$	$X=0$	1	2	3	4	5	6=M
0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1
2	0	0	1	2	2	3	3
3	0	0	1	2	5	5	6

$$f_3(X) = \begin{cases} -\infty & X < 0 \\ 0 & 0 \leq X < 2 \\ 1 & 2 \leq X < 3 \\ 2 & 3 \leq X < 4 \\ 5 & 4 \leq X < 6 \\ 6 & 6 \leq X < 7 \end{cases}$$

最优解

备忘录的大小与 n 和 M 有关。

回溯确定X

- 实例： $n=3$, $(w_1, w_2, w_3)=(2, 3, 4)$, $(p_1, p_2, p_3)=(1, 2, 5)$, $M=6$
- x_3 : $M=X=6$, $F(3,6) \neq F(2,6)$, 则 $x_3=1$, $X-w_3=6-4=2$
- x_2 : $F(2,2)=F(1,2)$, 则 $x_2=0$
- x_1 : $F(1,2) \neq F(0,2)$, 则 $x_1=1$

$F(i, X)$	$X=0$	1	2	3	4	5	6=M
0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1
2	0	0	1	2	2	3	3
3	0	0	1	2	5	5	6

当前不需要
标记函数。

思考： $M > \sum_{1 \leq i \leq n} w_i$ 时，备忘录有必记录到M列么？

算法6.2 0/1背包算法描述



```
procedure KNAPSACK(p,w,n,M)
```

```
int p(n), w(n), F(0:n,0:M), x(1:n), i, j, n//F是备忘录
```

```
for j  $\leftarrow$  0 to M do F(0,j)  $\leftarrow$  0 repeat
```

```
for i  $\leftarrow$  1 to n do
```

```
    for j  $\leftarrow$  0 to M do
```

```
        F(i,j)  $\leftarrow$  F(i-1,j)
```

```
        if w(i)  $\leq$  j and F(i,j) < F(i-1,j-w(i))+p(i) then
```

```
            F(i,j)  $\leftarrow$  F(i-1,j-w(i))+p(i)
```

```
        endif
```

```
    repeat
```

```
repeat //最优解在F(n,M)中
```

```
回溯确定x的值
```

```
end FGRAPH
```

- 时间复杂度: $\Theta(nM)$

- n是物品个数

- M是背包容量

- 空间复杂度: $\Theta(nM)$

伪多项式时间

思考: 可以认为0/1背包问题是多项式时间的算法么?

递归关系式图解

$$f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X-w_i)+p_i \}$$

$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

- $f_{i-1}(X-w_i)+p_i$ 的图像可由 $f_{i-1}(X)$ 在 x 轴上右移 w_i 个单位，然后上移 p_i 个单位得到。
- $f_i(X)$ 的图像由 $f_{i-1}(X)$ 和 $f_{i-1}(X-w_i)+p_i$ 的函数曲线按 X 相同时取最大值的规则归并而成。



实例: $n=3$, $(w_1, w_2, w_3)=(2, 3, 4)$, $(p_1, p_2, p_3)=(1, 2, 5)$, $M=6$

$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

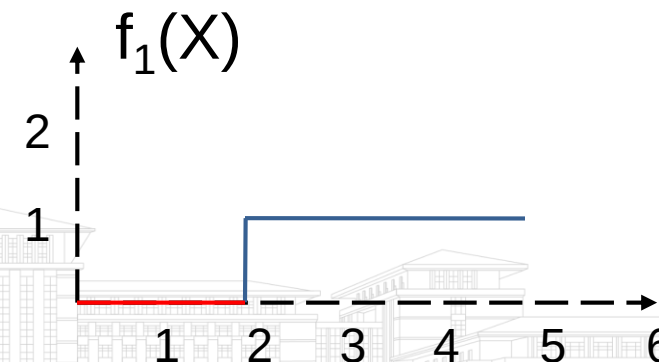
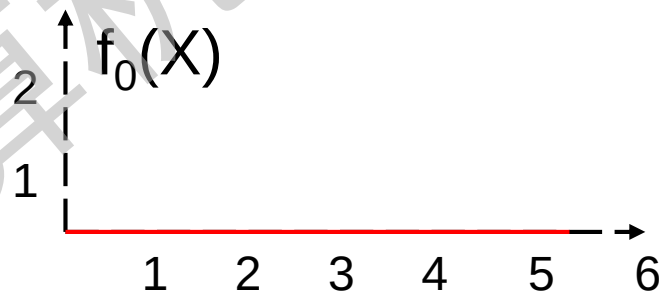
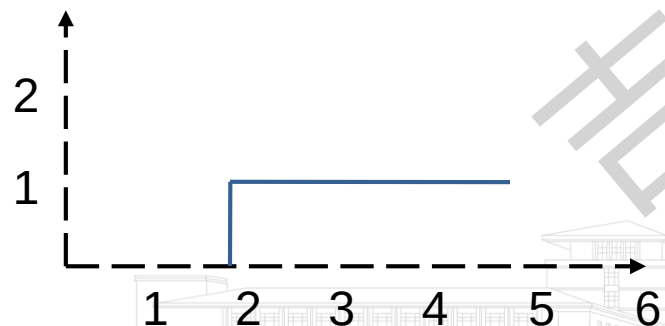
$$f_1(X) = \begin{cases} -\infty & X < 0 \\ \max\{0, -\infty + 1\} = 0 & 0 \leq X < 2 \\ \max\{0, 0 + 1\} = 1 & X \geq 2 \end{cases}$$

$$f_{i-1}(X-w_i)+p_i$$

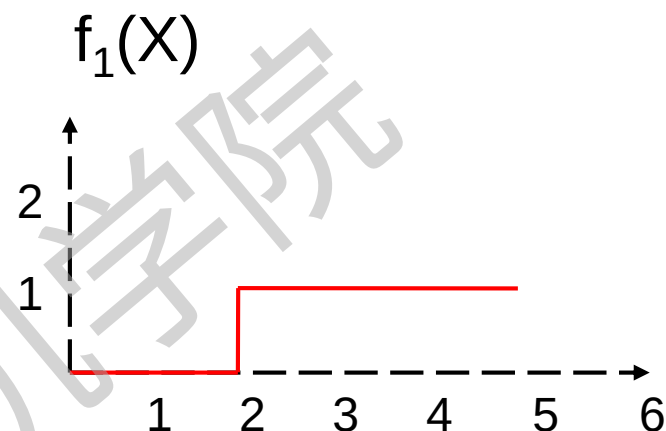
$$f_i(X)$$

$i=0$: 函数不存在

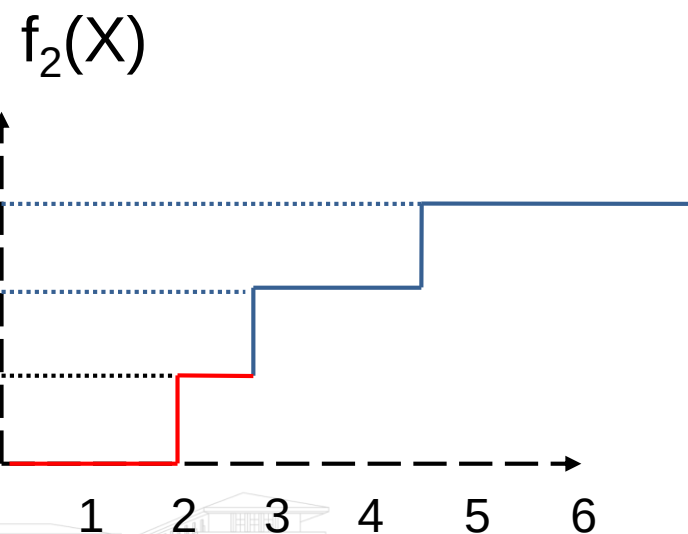
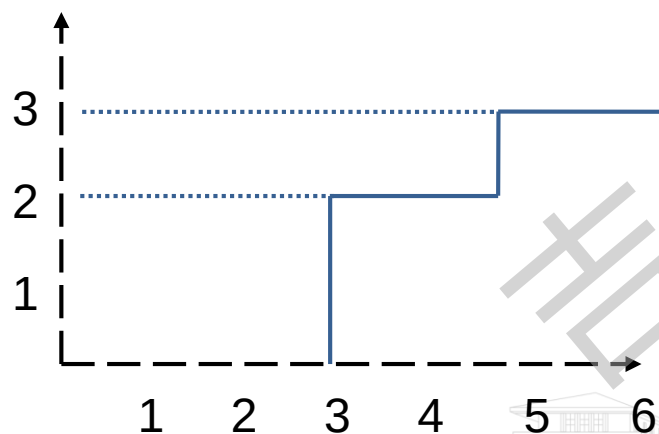
$$i=1: f_0(X-2)+1$$



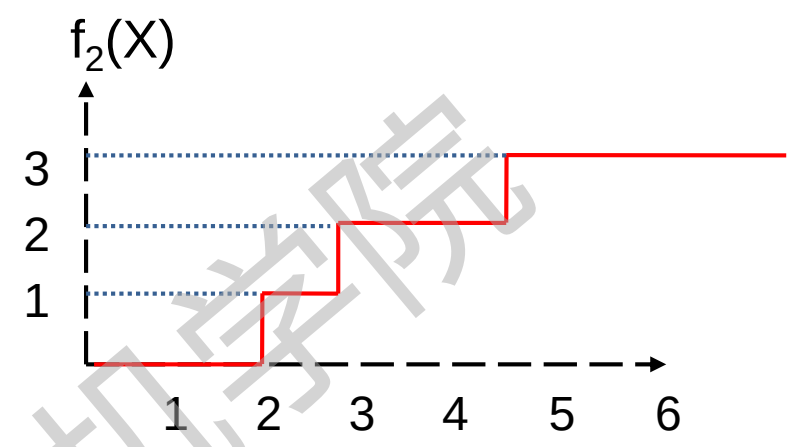
$$f_2(X) = \begin{cases} -\infty & X < 0 \\ 0 & 0 \leq X < 2 \\ 1 & 2 \leq X < 3 \\ \max\{1, 0+2\}=2 & 3 \leq X < 5 \\ \max\{1, 1+2\}=3 & 5 \leq X \leq 6 \end{cases}$$



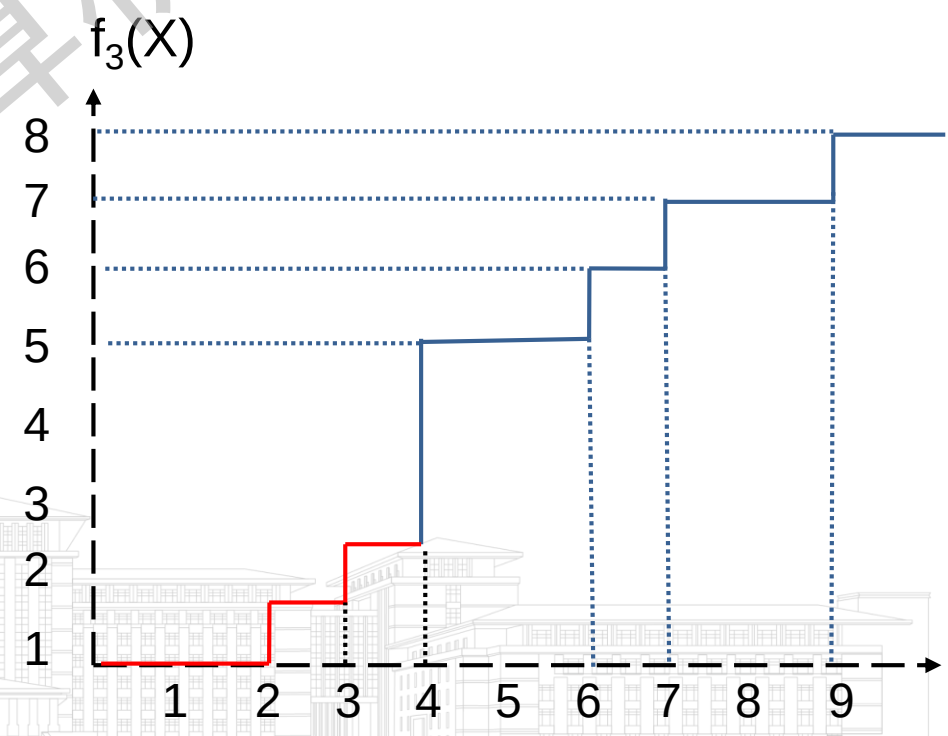
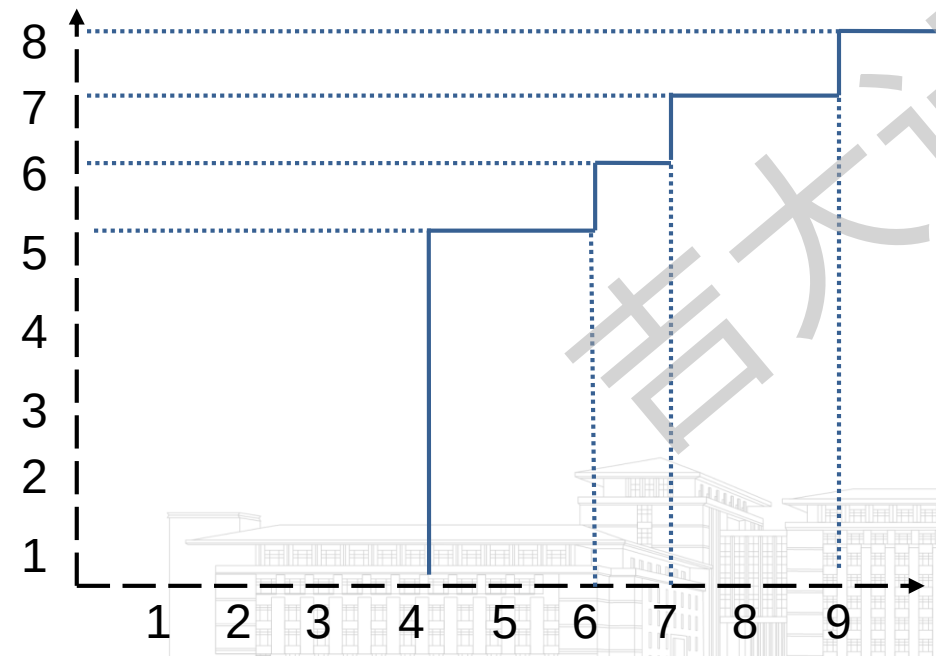
$i=2: f_1(X-3)+2$



$$f_3(X) = \begin{cases} -\infty & X < 0 \\ \max\{0, -\infty+5\} = 0 & 0 \leq X < 2 \\ \max\{1, -\infty+5\} = 1 & 2 \leq X < 3 \\ \max\{2, -\infty+5\} = 2 & 3 \leq X < 4 \\ \max\{2, 0+5\} = 5 & 4 \leq X < 5 \\ \max\{3, 0+5\} = 5 & 5 \leq X < 6 \\ \max\{3, 1+5\} = 6 & 6 \leq X < 7 \\ \max\{3, 2+5\} = 7 & 7 \leq X < 9 \\ \max\{3, 3+5\} = 8 & X \geq 9 \end{cases}$$

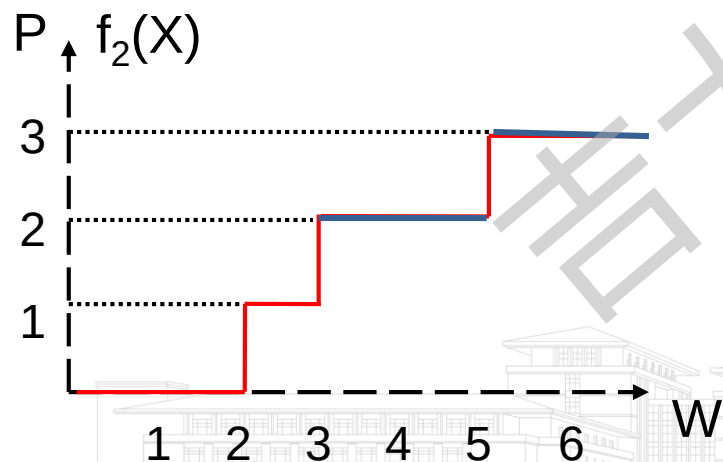


$$i=3: f_2(X-4)+5$$



函数曲线分析

- 每一个 f_i 完全由一些序偶 (P_j, W_j) 组成的集合所说明, 其中 W_j 是使 f_i 在其处产生一次阶跃的 X 值, $P_j = f_i(W_j)$ 。
- 第一对序偶 $(P_0, W_0) = (0, 0)$; 如果有 r 次阶跃, 就还有 r 对序偶 (P_j, W_j) , $1 \leq j \leq r$; 在 r 对序偶中, 假定 $W_j < W_{j+1}$, 则有 $P_j < P_{j+1}$ 。
- 在 $0 \leq j < r$ 情况下, 对所有满足 $W_j \leq X < W_{j+1}$ 的 X , 有 $f_i(X) = f_i(W_j)$; 对所有满足 $X \geq W_r$ 的 X , 有 $f_i(X) = f_i(W_r)$ 。



3次阶跃, f_2 由序偶 $(0,0), (1,2), (2,3), (3,5)$ 组成的集合所刻画

$3 \leq X < 5$, $f_i(X) = f_i(3) = 2$

$X \geq 5$, 有 $f_i(X) = f_i(5) = 3$

序偶对集合定义

- S^{i-1} : f_{i-1} 的所有序偶的集合
- S_1^i : $f_{i-1}(X-w_i)+p_i$ 的所有序偶的集合, 将 (p_i, w_i) 加到 S^{i-1} 的每一对序偶上就得到 S_1^i
- S^i : S^{i-1} 和 S_1^i 基于支配规则实现归并

支配规则: 已知 $(P_j, W_j) \in S^{i-1}$, $(P_k, W_k) \in S_1^i$, 并且在 $W_j \geq W_k$ 的同时有 $P_j \leq P_k$, 那么序偶 (P_j, W_j) 被放弃, 称为 (P_k, W_k) 支配 (P_j, W_j) 。



序偶对法设计思想

- $S^0 = \{(0,0)\}$
- $i \geq 1$ 时, 将 (p_i, w_i) 加到 S^{i-1} 的每一对序偶上就得到 S_1^i ,
即 $S_1^i = \{(p, w) | (p - p_i, w - w_i) \in S^{i-1}\}$
- 在支配规则下将 S^{i-1} 和 S_1^i 归并成 S^i 。
- 在生成 S^i 时, 将 $w > M$ 的那些序偶 (p, w) 去掉, 它们不能导出满足约束条件的可行解。

S^i 中的序偶对表示对于不同的约束 $X \leq M$, 前 i 个物品可能取得的最大效益值。

S^n 中最末序偶对的 P 值, 是问题的最优解。



序偶对法实例

实例: $n=3, (p_1, p_2, p_3)=(1, 2, 5), (w_1, w_2, w_3)=(2, 3, 4), M=6$

$$S^0=\{(0,0)\} \xrightarrow{(p_1, w_1)=(1,2)} S^1_1=\{(1,2)\}$$

$$S^1=\{(0,0), (1,2)\} \xrightarrow{(p_2, w_2)=(2,3)} S^2_1=\{(2,3), (3,5)\}$$

$w > M$, 舍弃掉

$$S^2=\{(0,0), (1,2), (2,3), (3,5)\} \xrightarrow{(p_3, w_3)=(5,4)} S^3_1=\{(5,4), (6,6), (7,7), (8,9)\}$$

$$S^3=\{(0,0), (1,2), (2,3), (5,4), (6,6)\}$$

根据支配规则, $(5,4)$ 支配掉 $(3,5)$

		0		1	2	3	4	5	6
F(i,j)	j	0	1	2	3	4	5	6	
	i=3	0	0	1	2	5	5	6	

思考: 对比F(i,j), 发现数值之间有什么关系?

确定决策序列

- 对于 S^n 的最末序偶 (p_l, w_l) , 若 $(p_l, w_l) \in S^{n-1}$, 则 $x_n=0$; 若 $(p_l, w_l) \notin S^{n-1}$, 且 $(p_l - p_n, w_l - w_n) \in S^{n-1}$, 则 $x_n=1$
- 再判断留在 S^{n-1} 的序偶 (p_l, w_l) 或 $(p_l - p_n, w_l - w_n)$ 是否属于 S^{n-2} 以确定 x_{n-1} 的取值。...

$$S^0 = \{(0,0)\}$$

$$S^1 = \{(0,0), (1,2)\} \quad x_1=1$$

$$S^2 = \{(0,0), (1,2), (2,3), (3,5)\} \quad x_2=0$$

$$S^3 = \{(0,0), (1,2), (2,3), (5,4), (6,6)\} \quad x_3=1$$

最优解 $f_n(M)$ 的最优决策序列是 $(x_1, x_2, x_3) = (1, 0, 1)$



序偶对法的数据结构

- $p(1:n)$: 物品的效益值。
- $w(1:n)$: 物品的质量。
- $P(1:m)$ 和 $W(1:m)$: 序偶对 (p_i, w_i) , 模拟序偶集合 S ; 序偶集 $S^0, S^1, \dots, S^{n-1}$ 互相邻接的存放。
- $F(0:n)$: 指针数组, 元素 $F(i)$ 指示 S^i 中的第一个元素所在的位置。

$$S^0 = \{(0,0)\}$$

$$S^1 = \{(0,0), (1,2)\}$$

$$S^2 = \{(0,0), (1,2), (2,3), (3,5)\}$$

	1	2	3	4	5	6	7	8
P	0	0	1	0	1	2	3	
W	0	0	2	0	2	3	5	

$\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$
 $F(0)$ $F(1)$ $F(2)$ $F(3)$



序偶对法的空间复杂度分析

- S^i 的序偶数量用 $|S^i|$ 表示,
- 每个 S^i 由 S^{i-1} 和 S^i_1 归并而成, 且 $|S^i_1| \leq |S^{i-1}|$, 故 $|S^i| \leq 2|S^{i-1}|$
- 最坏情况下, 归并过程中没有序偶被清除, 则

$$|S^0|=1, \quad |S^1|=2, \quad |S^2|=2+2=2^2, \quad \dots, \quad |S^{n-1}|=2^{n-1}$$

$$\sum_{0 \leq i \leq n-1} |S^i| = \sum_{0 \leq i \leq n-1} 2^i = 2^n - 1$$

- 则空间复杂度是 $O(2^n)$



序偶对法的时间复杂度分析

- 由 S^{i-1} 生成 S^i ，需要的时间是 $\Theta(|S^{i-1}|)$
- 计算所有的 $S^0, S^1, \dots, S^{n-1}$ ，需要的总时间为 $\Theta(\sum |S^{i-1}|)$
- 因为 $|S^i| \leq 2^i$ ，故总时间为 $O(2^n)$
- 若 P, W 均为整数，则有 $|S^i| \leq 1 + \sum_{1 \leq j \leq i} P_j$ ， $|S^i| \leq 1 + \min\{\sum_{1 \leq j \leq i} W_j, M\}$
- 故当 P 和 W 为整数时，时间复杂度为

$$O(\min\{2^n, n \sum_{1 \leq j \leq n} P_j, nM\})$$

M 一般远小于 2^n ，再加上支配规则，能解决 n 较大时的问題。

思考：比较迭代法和序偶对法的时间复杂度？



序偶对法的改进

- 试探法
 - L : 最优解的估计值, $f_n(M) \geq L$
 - $PLEFT(i) = \sum_{i < j \leq n} p_j$, 如果 $(p, w) \in S^i$, 且 $p + PLEFT(i) < L$, 那么 (p, w) 从 S^i 中清除掉

● 实例: $n=6, M=165$

i	1	2	3	4	5	6
p	100	50	20	10	7	3
w	100	50	20	10	7	3

	0	1	2	3	4	5	6
$PLEFT(i)$	190	90	40	20	10	3	0

- 贪心方法求出估计值 $L=163$
- $S^0 = \{0\}, S^1_1 = \{100\}$
- $S^1 = \{0, 100\}, S^2_1 = \{150\}$
- $S^2 = \{100, 150\}, S^3_1 = \emptyset$
- $S^3 = \{150\}, S^4_1 = \{160\}$
- $S^4 = \{150, 160\}, S^5_1 = \emptyset$
- $S^5 = \{160\}, S^6_1 = \{163\}$
- $S^6 = \{163\}$



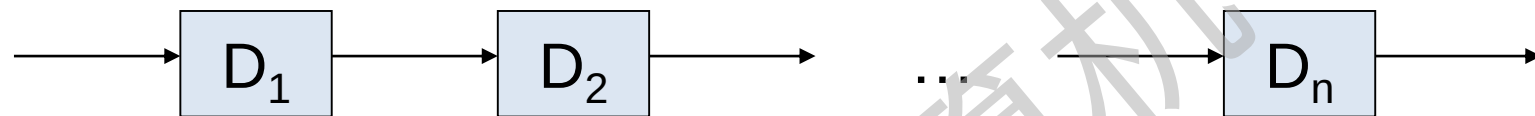
6.5 可靠性设计

- 问题描述
- 可靠性设计最优化问题
- 递推关系式
- 可靠性算法设计思想
- 问题实例计算过程



问题描述

- 假定设计一个系统，该系统由若干个以串联方式连接在一起的不同设备所组成。



- 设 r_i 是设备 D_i 正常运转的概率，即可靠性。

$\prod r_i$ 是整个系统的可靠性。

可靠性数据对比：

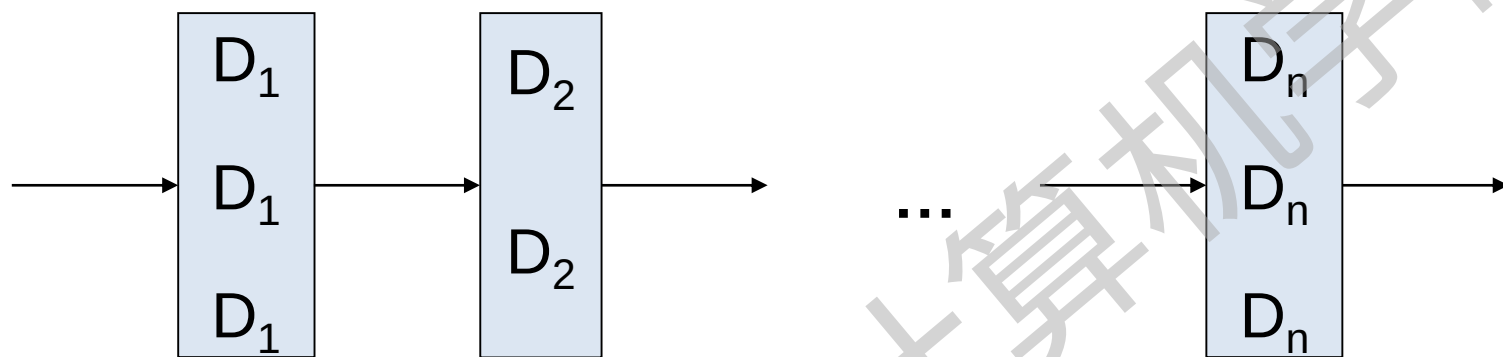
$$r_i=0.99, i=1,2,\dots,10 \quad \prod r_i = 0.99^{10} = 0.904$$

$$r_1=0.9, r_2=0.8, r_3=0.5, \quad \prod r_i = 0.36$$



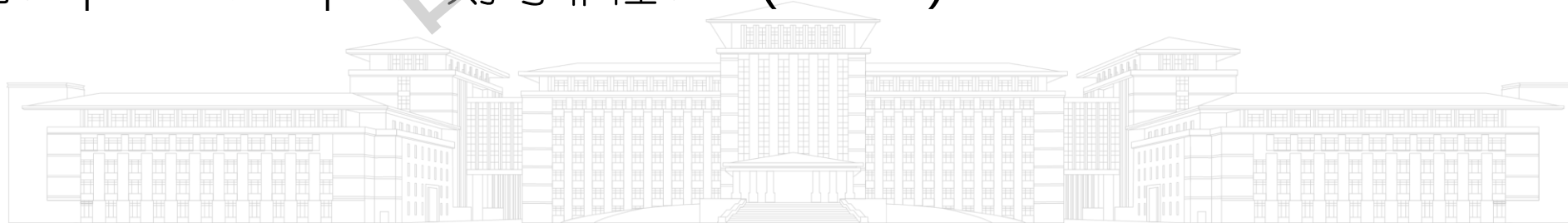
改进方法

- 增加一些重复的设备。



- 假设设备 D_i 的台数为 m_i ，那么它们同时出现故障的概率为 $(1-r_i)^{m_i}$ ，则可靠性变为 $1-(1-r_i)^{m_i}$ 。

例如： $r_i=0.99$ ， $m_i=2$ ， 则可靠性： $1-(1-0.99)^2=0.9999$



可靠性设计最优化问题

- 问题描述：在最大成本 c 的约束下，如何使系统的可靠性达到最优的问题。
- 已知系统中每种设备至少有一台，设 c_j 是一台设备 j 的成本， j 允许配置的台数至多为：

$$u_j = \left\lfloor (c + c_j - \sum_{k=1..n} c_k) / c_j \right\rfloor$$

- 设第 i 级的可靠性表示为函数： $\phi_i(m_i) = 1 - (1 - r_i)^{m_i}$



递推关系式

- 用 $RELI(1, i, X)$ 表示在可容许成本 X 约束下，对第 1 种到第 i 种设备的可靠性设计问题。

$$\begin{array}{ll} \text{极大化} & \prod_{1 \leq j \leq i} \phi_j(m_j) \\ \text{约束条件} & \sum_{1 \leq j \leq i} c_j m_j \leq X \quad 1 \leq m_j \leq u_j \end{array}$$

- $f_n(c) = \max_{1 \leq m_n \leq u_n} \{ \phi_n(m_n) f_{n-1}(c - c_n m_n) \}$
- $f_i(x) = \max_{1 \leq m_i \leq u_i} \{ \phi_i(m_i) f_{i-1}(c - c_i m_i) \}$
- 使用类似 0/1 背包问题的序偶对方法来求解



可靠性算法设计思想



支配规则:对于 (f_1, x_1) 和 (f_2, x_2) , 当且仅当 $f_1 \geq f_2$ 而 $x_1 \leq x_2$ 时, (f_1, x_1) 支配 (f_2, x_2) 。
 (f_2, x_2) 从序偶集合中舍去。

- 令 $S^0 = \{(1, 0)\}$

- 基于 S^{i-1} 构造 S_j^i :

对于 $(f, x) \in S^{i-1}$, 有
 $(\phi_i(j) \times f, x + c_i \times j) \in S_j^i$,
 $1 \leq j \leq u_i, x + c_i \times j + \sum_{i+1 \leq k \leq n} c_k \leq c_0$

- 基于支配规则归并:

$S^i = \bigcup_{1 \leq j \leq u_i} S_j^i$

求解 $u_i, i=1, 2, \dots, n$;

for $i \leftarrow 1$ to n

for $j \leftarrow 1$ to u_i

求 $\phi_i(j)$

求 S_j^i

repeat

基于支配规则归并 S^i

repeat



问题计算过程

- 设计一个由设备 D_1, D_2, D_3 组成的三级系统。每台设备的成本分别为30元，15元和20元，可靠性分别是0.9, 0.8, 0.5，计划建立该系统的投资不得超过105元。

$$c=105, c_1=30, c_2=15, c_3=20$$

$$r_1=0.9, r_2=0.8, r_3=0.5$$

- S^i : 表示 $m_1, \dots, m_i$ 的各种决策序列产生的所有不受支配的序偶 (f, x) 之集合。
- S_j^i : 表示第 i 级设备选择 j 台时 $m_1, \dots, m_{i-1}, j$ 的各种决策序列产生的所有不受支配的序偶 (f, x) 之集合。



- $S^0 = \{(1, 0)\}$
- $u_1 = \lfloor (105 - 15 - 20) / 30 \rfloor = 2, u_2 = 3, u_3 = 3$
- $u_1 = 2, c_1 = 30, r_1 = 0.9,$
 $\phi_1(1) = 0.9, \phi_1(2) = 1 - (1 - 0.9)^2 = 0.99$
 $S^1_1 = \{(0.9, 30)\}, S^1_2 = \{(0.99, 60)\}$
 $S^1 = \{(0.9, 30), (0.99, 60)\}$
- $u_2 = 3, c_2 = 15, r_2 = 0.8,$
 $\phi_2(1) = 0.8, \phi_2(2) = 1 - (1 - 0.8)^2 = 0.96, \phi_2(3) = 1 - (1 - 0.8)^3 = 0.992$
 $S^2_1 = \{(0.9 \times 0.8, 30 + 15), (0.99 \times 0.8, 60 + 15)\} = \{(0.72, 45), (0.792, 75)\}$
 $S^2_2 = \{(0.9 \times 0.96, 30 + 15 \times 2), (0.99 \times 0.96, \textcolor{red}{60 + 15 \times 2})\} = \{(0.864, 60)\}$
 $S^2_3 = \{(0.9 \times 0.992, 30 + 15 \times 3), (0.99 \times 0.992, \textcolor{red}{60 + 15 \times 3})\} = \{(0.8928, 75)\}$
 $S^2 = \{(0.72, 45), (\textcolor{red}{0.792, 75}), (0.864, 60), (0.8928, 75)\}$



- $S^2 = \{(0.72, 45), (0.864, 60), (0.8928, 75)\}$
- $u_3 = 3, c_3 = 20, r_3 = 0.5,$
 $\phi_3(1) = 0.5, \phi_3(2) = 1 - (1 - 0.5)^2 = 0.75, \phi_3(3) = 1 - (1 - 0.5)^3 = 0.875$
 $S^3_1 = \{(0.7 \times 0.5, 45 + 20), (0.864 \times 0.5, 60 + 20), (0.8928 \times 0.5, 75 + 20)\}$
 $= \{(0.36, 65), (0.432, 80), (0.4464, 95)\}$
 $S^3_2 = \{(0.72 \times 0.75, 45 + 20 \times 2), (0.864 \times 0.75, 60 + 20 \times 2),$
 $(0.8928 \times 0.75, ~~75 + 20 \times 2~~)\} = \{(0.54, 85), (0.648, 100)\}$
 $S^3_3 = \{(0.72 \times 0.875, 45 + 20 \times 3)\} = \{(0.63, 105)\}$
 $S^3 = \{(0.36, 65), (0.432, 80), (~~0.4464, 95~~), (0.54, 85), (0.648, 100), (~~0.63, 105~~)\}$
- 由 $S^n, S^{n-1} \dots S^1$ 回溯求 $m_n, m_{n-1}, \dots, m_1$
- 判断由 S^n 中 f 最大的序偶来自哪个 S^n_j , 则 n 级设备上的设备数量为 j , 以此类推:
- $(0.648, 100) \in S^3_2$, 所以 $m_3 = 2$; $(0.864, 60) \in S^2_2$, 所以 $m_2 = 2$; $(0.9, 30) \in S^1_1$, $m_1 = 1$
 决策序列 $(m_1, m_2, m_3) = (1, 2, 2)$



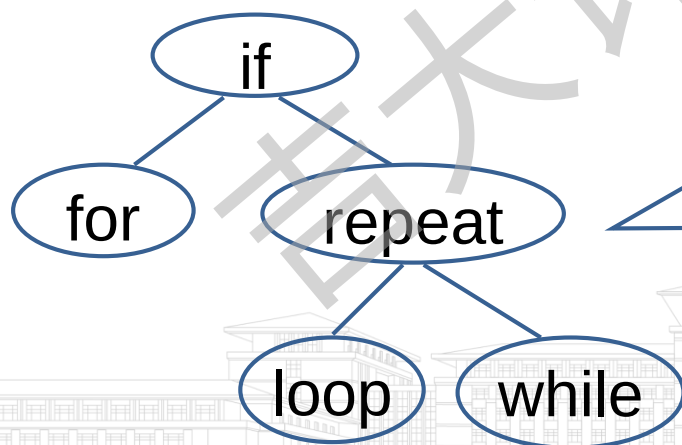
6.6 最优二分检索树

- 二分检索树
- 二分检索树检索算法
- 问题描述
- 二分检索树的预期成本
- 最优二分检索树
- 递推关系式分析
- 问题计算过程
- 时间复杂度分析
- D.E.Knuth的优化方法
- 最优二分检索树算法



二分检索树

- 二分检索树: T 是一个二叉树, 它或者为空, 或者其每个结点含有一个可比较大小的数据元素, 并且:
 - T 的左子树的所有元素比根结点 T 中的元素小
 - T 的右子树的所有元素比根结点 T 中的元素大
 - T 的左、右子树也是二分检索树



二分检索树中的所有结点的元素是互异的。

二分检索树检索算法

- 将X与根比较:
 - 若X比根中的元素小则检索左子树
 - 若X等于根中的元素则检索成功终止
 - 若X比根中的元素大则检索右子树

每个结点有3个信息段,
LCHILD, RCHILD, IDENT。
若X不在T中, 则i=0; 否则,
将i置成使得IDENT(i)=X。

```

procedure SEARCH(T, X, i)
    i ← T
    while i ≠ 0 do
        case
            :X < IDENT(i): i ← LCHILD(i)
            :X = IDENT(i): return
            :X > IDENT(i): i ← RCHILD(i)
        endcase
    repeat
end SEARCH
    
```



问题描述

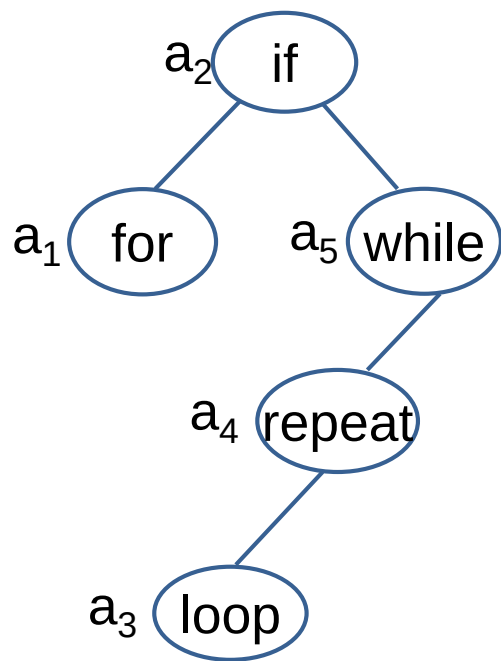
- 已知一个固定的标识符集合，希望产生一种构造二分检索树的方法，使平均检索次数**最小**，即构造一棵最优二分检索树。
- 二分检索树用圆结点(内结点)表示给出的 n 个标识符,对应 n 种成功的情况。方结点(外结点)表示不成功检索的情况,有 $n+1$ 个外结点。
- 同一个标识符集合的不同二分检索树有不同的性能特征。



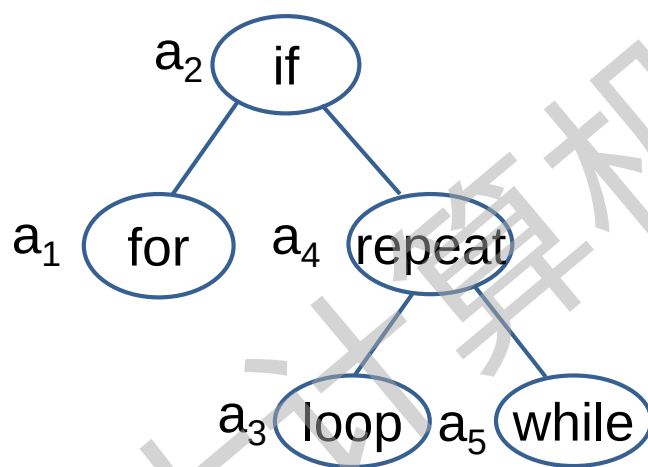
问题实例



- 标识符集 $(a_1, a_2, a_3, a_4, a_5) = (\text{for}, \text{if}, \text{loop}, \text{repeat}, \text{while})$



图a



图b

- 最坏情况下，图a需4次比较，图b需3次比较。
- 假设每个结点成功检索的概率是 $1/5$ ，则在平均成功情况下，图a需 $12/5$ 次比较，图b需 $11/5$ 次比较。

一般情况下，要检索的那些标识符具有不同的概率，而且存在不成功检索的情况。



二分检索树的预期成本

- 设给定的标识符集是 $(a_1, a_2, \dots, a_n)$, $a_1 < a_2 < \dots < a_n$, $P(i)$ 是对 $X = a_i$ 成功检索的概率, $Q(i)$ 是 $a_i < X < a_{i+1}$ 不成功检索的概率(假定 $a_0 = -\infty$, $a_{n+1} = +\infty$),

$$\sum_{1 \leq i \leq n} P(i) + \sum_{0 \leq i \leq n} Q(i) = 1$$

- 一次成功检索在一个内结点(l 级)处终止, 算法中的循环要执行 l 次, 所以内结点 a_i 的预期成本为 $P(i) * \text{level}(a_i)$
- 一次不成功检索在一个外结点处(l 级)终止, 算法中循环要执行 $l-1$ 次, 所以外结点 E_i 的预期成本为 $Q(i) * (\text{level}(E_i) - 1)$

$$\text{二分检索树的预期成本为: } \sum_{1 \leq i \leq n} P(i) * \text{level}(a_i) + \sum_{0 \leq i \leq n} Q(i) * (\text{level}(E_i) - 1)$$



最优二分检索树

- 标识符集($a_1, a_2, \dots, a_n$)的最优二分检索树是一棵使期望成本取最小值的树。

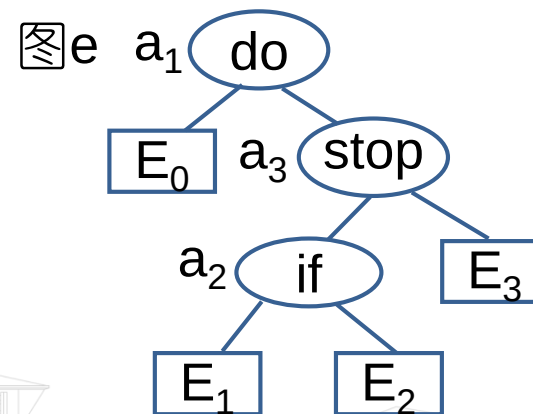
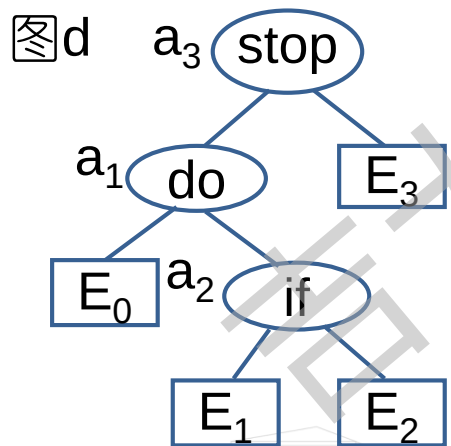
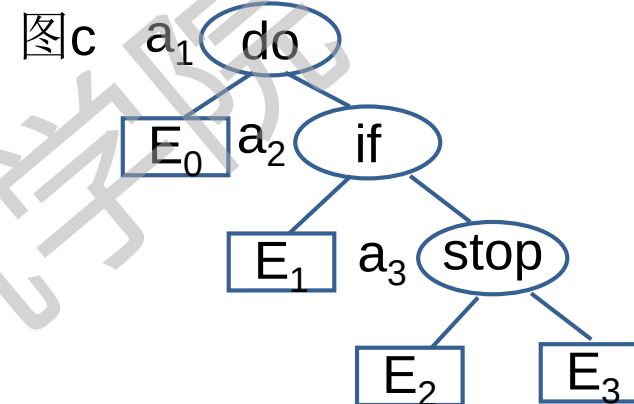
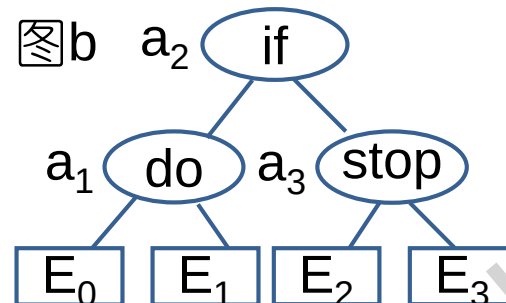
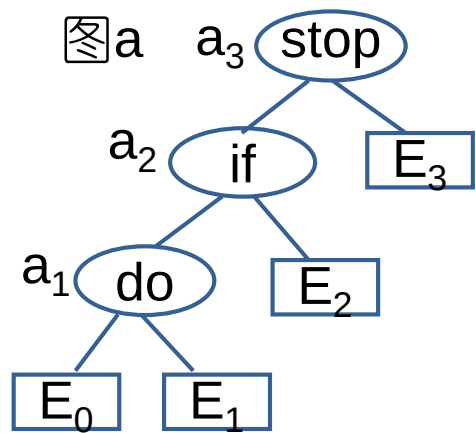
例: 标识符集(a_1, a_2, a_3)=(do, if, stop)的二分检索树中, 成功结点3个, 不成功结点4个。

情景1. 设内外结点概率相同: $P(i)=Q(i)=1/7$

情景2. 设内外结点概率不同: $P(i)=(0.5, 0.1, 0.05), Q(i)=(0.15, 0.1, 0.05, 0.05)$

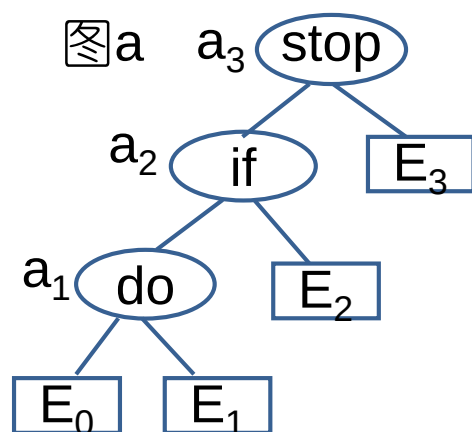


$n=3$ 时，有5种可能的二分检索树



情景1: $P(i)=Q(i)=1/7$

情景2: $P(i)=(0.5, 0.1, 0.05), Q(i)=(0.15, 0.1, 0.05, 0.05)$



$$\begin{aligned} \text{cost}(a) &= P(1)*3 + P(2)*2 + P(3)*1 \\ &\quad + Q(0)*3 + Q(1)*3 + Q(2)*2 + Q(3)*1 \end{aligned}$$

$$\begin{aligned} \text{cost}(a-1) &= 3*1/7 + 2*1/7 + 1/7 + 3*1/7 + 3*1/7 + 2*1/7 + 1/7 \\ &= 15/7 \end{aligned}$$

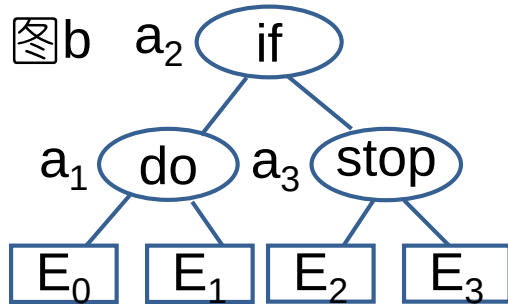
$$\begin{aligned} \text{cost}(a-2) &= 3*0.5 + 2*0.1 + 0.05 + 3*0.15 + 3*0.1 + 2*0.05 + 0.05 \\ &= 2.65 \end{aligned}$$



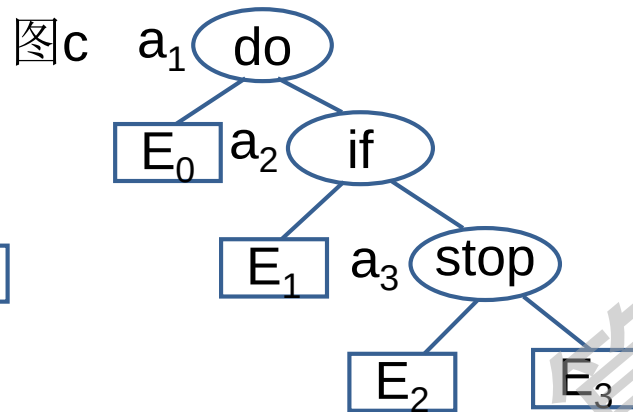
cost(a-1)=15/7
cost(a-2)=2.65

情景1: $P(i)=Q(i)=1/7$

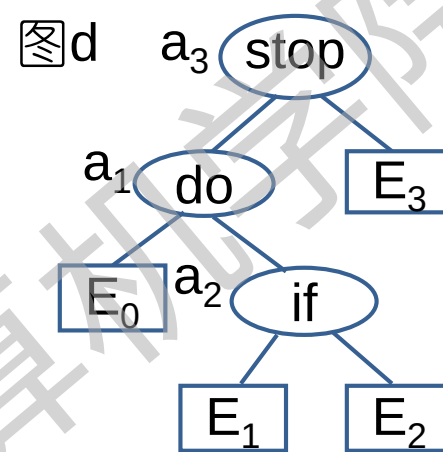
情景2: $P(i)=(0.5, 0.1, 0.05), Q(i)=(0.15, 0.1, 0.05, 0.05)$



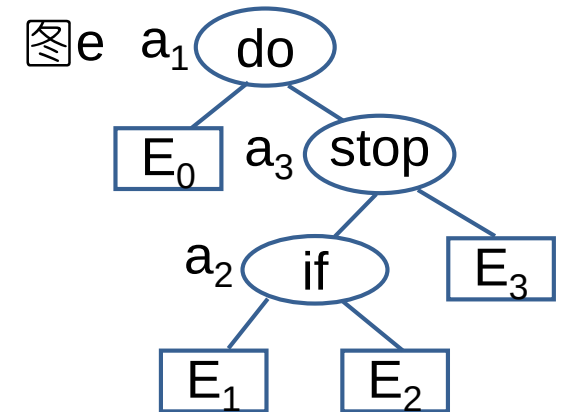
cost(b-1)=13/7
cost(b-2)=1.9



cost(c-1)=15/7
cost(c-2)=1.5



cost(d-1)=15/7
cost(d-2)=2.15



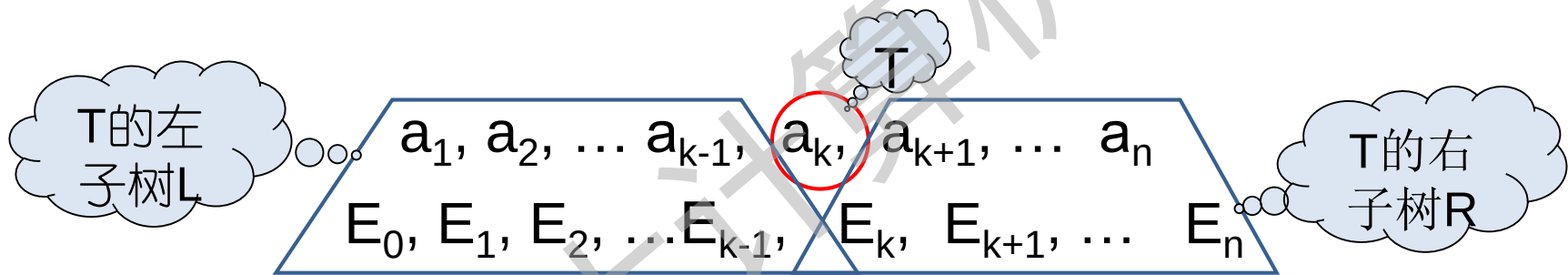
cost(e-1)=15/7
cost(e-2)=1.6

● 比较五种树形结构

- 对于第1种情况, 内外结点概率相同, 图b所示的树最优。
- 对于第2种情况, 内外结点概率不同, 图c所示的树最优。

递推关系式分析

- 为了把动态规划方法应用于得到一棵最优二分检索树的问题，需要把构造这样的一棵树看成是一系列决策的结果，而且要列出求取最优决策序列的递推式。
- 对于 $a_i, 1 \leq i \leq n$ ，决策出哪一个作为 T 的根结点。



左子树的成本:
$$\text{COST}(L) = \sum_{1 \leq i < k} P(i) * \text{level}(a_i) + \sum_{0 \leq i < k} Q(i) * (\text{level}(E_i) - 1)$$

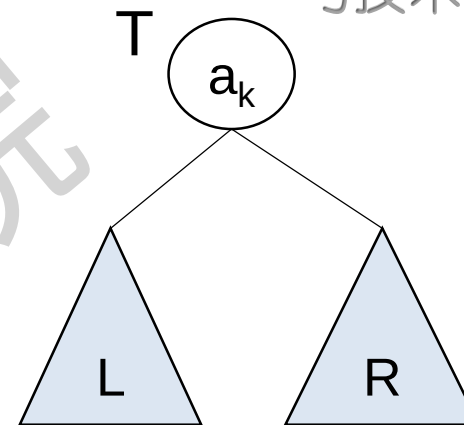
右子树的成本:
$$\text{COST}(R) = \sum_{k < i \leq n} P(i) * \text{level}(a_i) + \sum_{k \leq i \leq n} Q(i) * (\text{level}(E_i) - 1)$$

独立树

$$\text{COST}(T) = \sum_{1 \leq i \leq n} P(i) * \text{level}(a_i) + \sum_{0 \leq i \leq n} Q(i) * (\text{level}(E_i) - 1)$$

$$= \text{COST}(L) + \text{COST}(R) + P(k)$$

$$W(0, k-1) = \sum_{1 \leq i < k} P(i) + \sum_{0 \leq i < k} Q(i) + \sum_{k < i \leq n} P(i) + \sum_{k \leq i \leq n} Q(i) = W(k, n)$$



令 $W(i, j) = Q(i) + \sum_{l=i+1}^j (Q(l) + P(l))$, 且有 $W(i, j) = W(i, j-1) + Q(j) + P(j)$

树T的预期成本可由左右子树的预期成本表达:

$$\text{COST}(T) = P(k) + \text{COST}(L) + \text{COST}(R) + W(0, k-1) + W(k, n)$$



动态规划求解思想



- 证明满足最优性原理:
 - 若当前二分检索树 T 是最优的,则它的左右子树也一定是它们所对应的子问题的最优解
- 设计递推式:
 - 问题最优解: 最小期望成本
 - 问题边界和约束条件: 标识符范围 i, j
 - 原问题和子问题关系: 见最优性原理
- 自底向上方式实现程序
 - 求 $j-i=1$ 时的最优二分检索树
 - 求 $j-i=2$ 时的...
 - ...
 - 求 $j-i=n$ 时的最优二分检索树

递推关系式

- $C(i,j)$ 表示包含 $a_{i+1}, \dots, a_j$ 和 $E_i, \dots, E_j$ 的最优二分检索树的成本
- 根据最优性原理, 如果 T 是最优的, 那么 $COST(T)$ 必定是最小值, $COST(L)$ 和 $COST(R)$ 也必定是最小值。既 $COST(T)=C(0,n)$, $COST(L)=C(0,k-1)$, $COST(R)=C(k,n)$

每个根结点考察一遍

- 树 T 的最小预期成本:

$$C(0,n)=\min_{1 \leq k \leq n} \{C(0, k-1)+C(k, n)+P(k)+W(0, k-1)+W(k, n)\}$$

- 将上式一般化, 对于任何 $c(i,j)$ 有下式:

$$C(i,j)=\min_{i < k \leq j} \{C(i, k-1)+C(k, j)+P(k)+W(i, k-1)+W(k, j)\}$$

$$=\min_{i < k \leq j} \{C(i, k-1)+C(k, j)\}+W(i, j)$$

问题计算过程

$$C(i,j)=\min_{i \leq k \leq j} \{C(i, k-1)+C(k, j)\}+W(i, j)$$

- 考虑初始时，内结点个数为0个， $j-i=0$: $C(i, i)=0$, $W(i, i)=Q(i)$, $0 \leq i \leq n$
- 基于公式计算 $j-i=1$ 的 $C(i, j)$, $j-i=2$ 的 $C(i, j)$, ..., $j-i=n$ 的 $C(i, j)$
- 记下每棵树 T_{ij} 的根 $R(i,j)$, $R(i,j)$ 是使 $C(i,j)$ 取最小值时的 k 值

例: $n=4, (a_1, a_2, a_3, a_4) = (\text{do}, \text{if}, \text{read}, \text{while})$

$P(1:4) = (3, 3, 1, 1)$

$Q(0:4) = (2, 3, 1, 1, 1)$

(0) 初始时, $j-i=0$: $C(i, i)=0$, $W(i, i)=Q(i)$, $0 \leq i \leq 4$



$P(1:4)=(3,3,1,1)$
 $Q(0:4)=(2,3,1,1,1)$

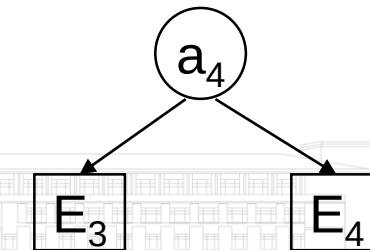
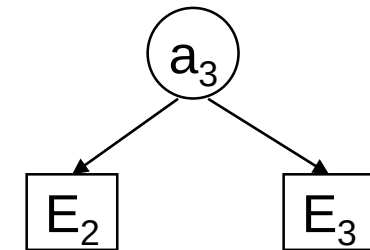
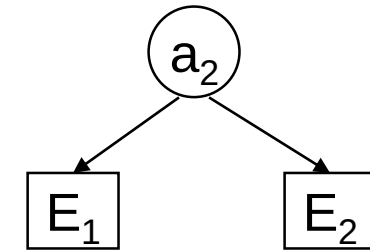
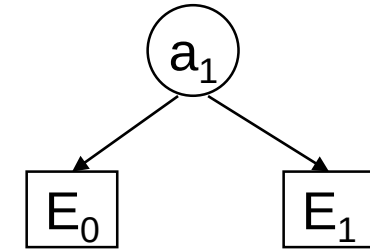
(1) $j-i=1$ 时, $0 \leq i < 4$

$C(0,1)$ $W(0,1)=Q(0)+Q(1)+P(1)=2+3+3=8$
 $C(0,1)=\min_{0 \leq k \leq 1} \{C(0,0)+C(1,1)\}+W(0,1)=0+8=8$
 $R(0,1)=k=1$

$C(1,2)$ $W(1,2)=Q(1)+Q(2)+P(2)=3+1+3=7$
 $C(1,2)=\min_{1 \leq k \leq 2} \{C(1,1)+C(2,2)\}+W(1,2)=0+7=7$
 $R(1,2)=k=2$

$C(2,3)$ $W(2,3)=3$
 $C(2,3)=3$
 $R(2,3)=k=3$

$C(3,4)$ $W(3,4)=3$
 $C(3,4)=3$
 $R(3,4)=k=4$



$W(0,1)=8$	$C(0,1)=8$	$R(0,1)=1$
$W(1,2)=7$	$C(1,2)=7$	$R(1,2)=2$
$W(2,3)=3$	$C(2,3)=3$	$R(2,3)=3$
$W(3,4)=3$	$C(3,4)=3$	$R(3,4)=4$

$$P(1:4)=(3,3,1,1)$$

$$Q(0:4)=(2,3,1,1,1)$$

(2) $j-i=2$ 时, $0 \leq i < 3$

C(0,2) $W(0,2)=W(0,1)+Q(2)+P(2)=8+1+3=12$

$$C(0,2)=\min_{0 < k \leq 2} \{ \underbrace{C(0,0)}_{k=1} + \underbrace{C(1,2)}_{k=2}, \underbrace{C(0,1)}_{k=1} + \underbrace{C(2,2)}_{k=2} \} + W(0,2)$$

$$= \min\{0+7, 8+0\} + 12 = 19$$

$$R(0,2)=k=1$$

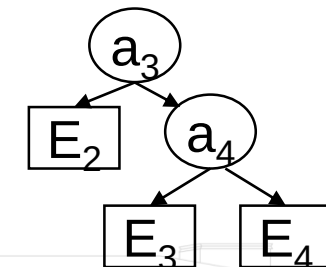
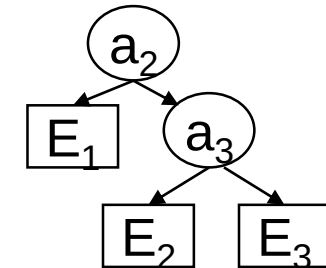
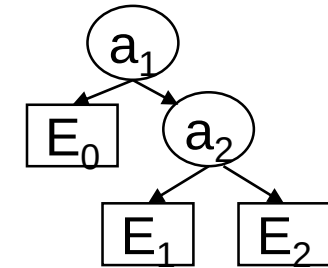
C(1,3) $W(1,3)=W(1,2)+Q(3)+P(3)=7+1+1=9$

$$C(1,3)=\min_{1 < k \leq 3} \{ \underbrace{C(1,1)}_{k=2} + \underbrace{C(2,3)}_{k=3}, \underbrace{C(1,2)}_{k=2} + \underbrace{C(3,3)}_{k=3} \} + W(1,3)$$

$$= \min\{0+3, 7+0\} + 9 = 12$$

$$R(1,3)=k=2$$

C(2,4) $W(2,4)=5 \quad C(2,4)=8 \quad R(2,4)=k=3$



$$\begin{array}{lll} W(0,1)=8 & C(0,1)=8 & R(0,1)=1 \\ W(1,2)=7 & C(1,2)=7 & R(1,2)=2 \\ W(2,3)=3 & C(2,3)=3 & R(2,3)=3 \\ W(3,4)=3 & C(3,4)=3 & R(3,4)=4 \end{array}$$

$$\begin{array}{lll} W(0,2)=12 & C(0,2)=19 & R(0,2)=1 \\ W(1,3)=9 & C(1,3)=12 & R(1,3)=2 \\ W(2,4)=5 & C(2,4)=8 & R(2,4)=3 \end{array}$$

$$\begin{array}{l} P(1:4)=(3,3,1,1) \\ Q(0:4)=(2,3,1,1,1) \end{array}$$

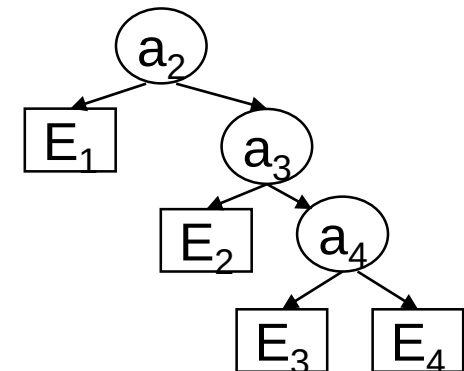
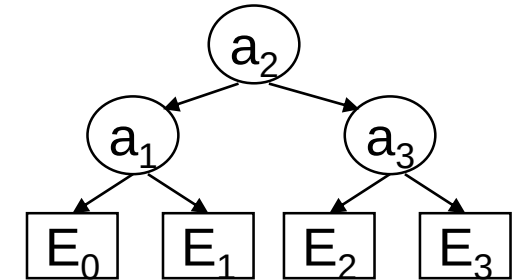
(3) $j-i=3$ 时, $0 \leq i < 2$

$$\begin{array}{l} C(0,3) \quad W(0,3)=W(0,2)+Q(3)+P(3) \\ \quad \quad \quad =12+1+1=14 \end{array}$$

$$\begin{array}{l} C(0,3)=\min \{ \underbrace{C(0,0)}_{0 < k \leq 3} + \underbrace{C(1,3)}_{k=1}, \underbrace{C(0,1)}_{0 < k \leq 3} + \underbrace{C(2,3)}_{k=2}, \underbrace{C(0,2)}_{0 < k \leq 3} + \underbrace{C(3,3)}_{k=3} \} + W(0,3) \\ \quad \quad \quad =\min\{0+12, 8+3, 19+0\}+14=11+14=25 \end{array}$$

$$R(0,3)=k=2$$

$$C(1,4) \quad W(1,4)=11 \quad C(1,4)=19 \quad R(1,4)=k=2$$



$W(0,1)=8$	$C(0,1)=8$	$R(0,1)=1$
$W(1,2)=7$	$C(1,2)=7$	$R(1,2)=2$
$W(2,3)=3$	$C(2,3)=3$	$R(2,3)=3$
$W(3,4)=3$	$C(3,4)=3$	$R(3,4)=4$

$W(0,2)=12$	$C(0,2)=19$	$R(0,2)=1$
$W(1,3)=9$	$C(1,3)=12$	$R(1,3)=2$
$W(2,4)=5$	$C(2,4)=8$	$R(2,4)=3$

$W(0,3)=14$	$C(0,3)=25$	$R(0,3)=2$
$W(1,4)=11$	$C(1,4)=19$	$R(1,4)=2$

$P(1:4)=(3,3,1,1)$
 $Q(0:4)=(2,3,1,1,1)$

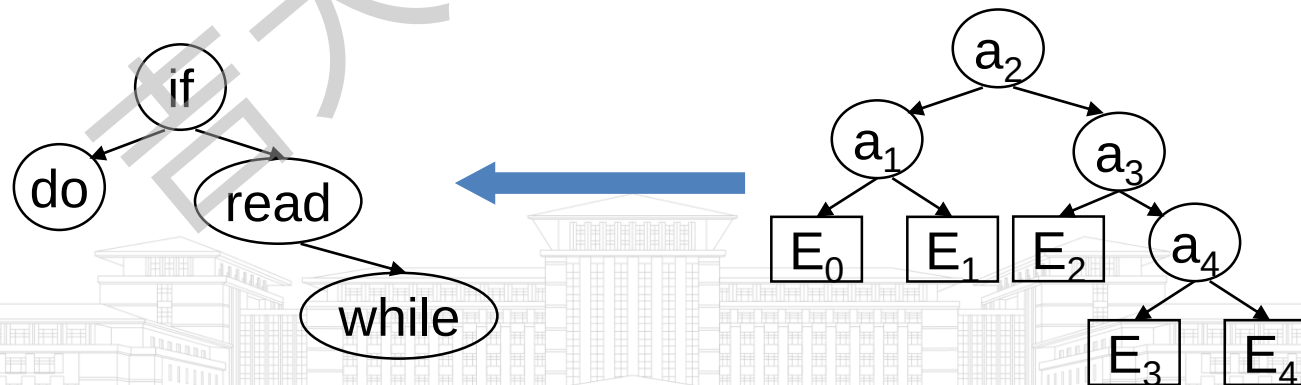
(4) $j-i=4$ 时, $0 \leq i < 1$

$$W(0,4)=W(0,3)+Q(4)+P(4)=16$$

$$C(0,4)=\min_{0 < k \leq 4} \{ \underbrace{C(0,0)}_{k=1} + \underbrace{C(1,4)}_{k=2}, \underbrace{C(0,1)}_{k=2} + \underbrace{C(2,4)}_{k=3}, \underbrace{C(0,2)}_{k=3} + \underbrace{C(3,4)}_{k=4}, \underbrace{C(0,3)}_{k=4} + \underbrace{C(4,4)}_{k=4} \} + W(0,4)$$

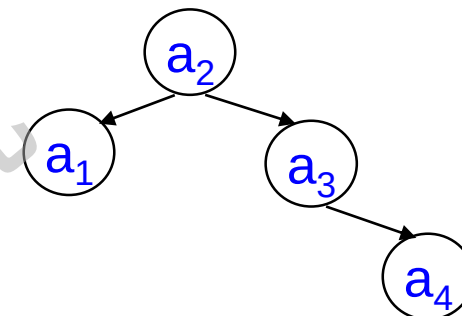
$$= \min\{0+19, 8+8, 19+3, 25+0\} + 16 = 16 + 16 = 32$$

$$R(0,4)=k=2$$





备忘录与标记函数



$W(i,j)$	$j=0$	1	2	3	4
0	2	8	12	14	16
1		3	7	9	11
2			1	3	5
3				1	3
4					1

$C(i,j)$	$j=0$	1	2	3	4
0	0	8	12	25	32
1		0	7	9	19
2			0	3	5
3				0	3
4					0

$R(i,j)$	$j=0$	1	2	3	4
0	0	1	1	2	2
1		0	2	2	2
2			0	3	3
3				0	4
4					0



时间复杂度分析

$$C(i,j)=\min_{i < k \leq j} \{C(i, k-1)+C(k, j)\}+W(i, j)$$

- $j-i=1$ 时,计算了 n 个 $C(i,j)$
- $j-i=2$ 时,计算了 $n-1$ 个 $C(i,j)$
- $j-i=m$ 时,计算了 $n-m+1$ 个 $C(i,j)$
- 为找出 m 个量中的最小值。 $C(i,j)$ 要在 $O(m)$ 时间内计算。
- 对于具有 $j-i=m$ 的所有 $C(i,j)$ 总的计算时间是 $O(nm-m^2)$ 。
- 对于所有的 $j-i=m$, $m=1,2,\dots,n$, 有: $\sum_{1 \leq m \leq n} (nm-m^2) = O(n^3)$



D.E.Knuth的优化方法

- 求最优的 k 可以通过把检索限制在区间 $R(i,j-1) \leq k \leq R(i+1,j)$ 求解递归关系式，这种情况下，计算时间为 $O(n^2)$ 。

$$C(i,j) = \min_{R(i,j-1) \leq k \leq R(i+1,j)} \{C(i, k-1) + C(k, j)\} + W(i, j)$$

初始值: $W(i,i) = Q(i)$; $W(i,i+1) = Q(i) + P(i+1) + Q(i+1)$;

$R(i,i) = 0$; $R(i,i+1) = i+1$;

$C(i,i) = 0$; $C(i,i+1) = Q(i) + P(i+1) + Q(i+1)$;



算法6.4 最优二分检索树算法

```

procedure OBST(P,Q,n)
real P(1:n), Q(0:n), C(0:n, 0:n), W(0:n, 0:n), integer R(0:n, 0:n),
for i←0 to n-1 do
    ( W(i,i), R(i,i), C(i,i)) ← (Q(i),0,0)
    (W(i,i+1), R(i,i+1) , C(i,i+1)) ←(Q(i)+Q(i+1)+P(i+1), i+1, Q(i)+Q(i+1)+P(i+1))
repeat
(W(n,n), R(n,n) , C(n,n) ) ←(Q(n),0,0)
for m← 2 to n do
    for i← 0 to n-m do
        j←i+m
        W(i,j) ← W(i,j-1)+P(j)+Q(j)
        k ← 区间  $[R(i,j-1), R(i+1,j)]$  中使  $\{C(i,l-1)+C(l,j)\}$  取最小值的  $l$  值
        C(i,j) ←C(i,k-1)+C(k,j)+W(i,j)
        R(i,j) ←k
    repeat
repeat
end OBST
    
```



6.7 矩阵链乘积问题

- 问题描述
- 问题分析
- 最优性原理证明
- 递推关系式设计
- 自底向上实现
- 矩阵链乘积算法
- 算法实例



问题描述

- 矩阵链乘积问题
 - 给定一个 n 个矩阵的序列，即矩阵链 $A_1, A_2, \dots, A_n$ ，计算它们的乘积 $A_1 A_2 \dots A_n$ ，使得代价最小，即所需的乘法次数最少。
- 乘法执行次数作为计算代价
 - 求矩阵 $A_{p \times q}$ 和 $B_{q \times r}$ 的乘积 $T_{p \times r}$ ，计算 $T_{p \times r}$ 中的一个元素，需要的乘法次数为 q 次，共 $p \times r$ 个元素，共需乘法次数为 $p \times q \times r$

$$T_{ij} = \sum_{k=1}^q A_{ik} B_{kj}$$



问题分析

- 对矩阵链乘积运算的顺序不同，代价也是不同的
 - 矩阵 $(A_1)_{p \times q}, (A_2)_{q \times r}, (A_3)_{r \times m}$
 - $(A_1 A_2) A_3$: $pqr + pr m$
 - $A_1 (A_2 A_3)$: $qrm + pqm$
 - 令 $p=10, q=100, r=5, m=50$ ，求出乘法次数
 - $(A_1 A_2) A_3$: 7500
 - $A_1 (A_2 A_3)$: 75000
- 对矩阵链加括号的方式不同，矩阵乘积运算的代价也不同

矩阵链乘积问题即寻找最优的括号方案，使得乘积运算的代价最小



问题分析

- 穷举法
- 动态规划法
 - 是不是多阶段决策最优问题？
 - 是否满足最优性原理？
 - 设计递推关系式
 - 自底向上实现

约束条件：矩阵乘积规则
目标函数：乘法次数最少

任意阶段决定括号位置，仅依赖于矩阵链当前状态，而之前如何到达当前状态无关。



最优性原理证明

●证明思想

- 要计算 $A_i A_{i+1} \dots A_j$ 的最小乘积代价，每次要决策分割点的位置，一旦确定位置，原问题被分割成两个子问题
- 假设最优解中，最外层的括号分割点在 A_k 后，即 $(A_i \dots A_k) (A_{k+1} \dots A_j)$ ，最优解所给出的方案对于子问题 $(A_i \dots A_k)$ 和 $(A_{k+1} \dots A_j)$ 也应该最优，否则与整个最优解的假设矛盾。



递推关系式设计

- 计算 $A_i A_{i+1} \dots A_j$ 的乘积, 其中 A_t 为 $p_{t-1} \times p_t$ 的矩阵, $t=i, \dots, j$
- 假设当前在 A_k 后分割, $i \leq k < j$, 原矩阵链被分割为两个矩阵链:

$$\begin{array}{c} A_i A_{i+1} \dots A_k \mid A_{k+1} \dots A_j \\ p_{i-1} \times p_k \mid p_k \times p_j \end{array}$$

乘积矩阵的大小

- 设 $m(i, j)$ 表示矩阵链 $A_i A_{i+1} \dots A_j$ 的最少代价, 则在 A_k 后分割的最少代价可以表示为:

$$m(i, k) + m(k+1, j) + p_{i-1} p_k p_j$$

- 考虑分割点 A_k , $i \leq k < j$, 则矩阵链 $A_i, \dots, A_j$ 的最少代价 $m(i, j)$ 是遍历 k 之后获得的最小值, 表示为:

$$m(i, j) = \begin{cases} 0, & i = j \\ \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1} p_k p_j\}, & i < j \end{cases}$$

自底向上实现

$$m(i, j) = \begin{cases} 0, & i = j \\ \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1}p_kp_j\}, & i < j \end{cases}$$

- 根据公式求解 $A_1, \dots, A_n$ 的最小代价，即求解 $m(1, n)$ ，步骤如下，其中 $1 \leq i, j \leq n$ ：
 - 计算 $j-i=1$ 时的所有 $m(i, j)$ ，共计 $n-1$ 个
 - 计算 $j-i=2$ 时的所有 $m(i, j)$ ，共计 $n-2$ 个
 - ...
 - 计算 $j-i=n-1$ 时的所有 $m(i, j)$ ，共计1个
- 问题得解

2个矩阵相乘



算法6.5 矩阵链乘积算法



Procedure MCO(n)

// A_1 为 $p_0 \times p_1$, A_2 为 $p_1 \times p_2$, ... A_n 为 $p_{n-1} \times p_n$

int $m(1:n, 1:n)$, $s(1:n-1, 1:n)$ //s记录分割点k

for $i=1$ to $n-1$ do //初始化

$(m(i,i), s(i,i)) \leftarrow (0, 0)$, $(m(i, i+1), s(i, i+1)) \leftarrow (p_{i-1} p_i p_{i+1}, i)$

repeat

$(m(n,n), s(n,n)) \leftarrow (0, 0)$

for $w \leftarrow 2$ to $n-1$ do //w表示问题规模：连乘的矩阵个数-1

for $i \leftarrow 1$ to $n-w$ do

$j \leftarrow i+w$

$k \leftarrow$ 区间 $[i, j]$ 中使 $\{m(i, l) + m(l+1, j) + p_{i-1} p_l p_j\}$ 取最小值的k

$m(i, j) \leftarrow m(i, k) + m(k+1, j) + p_{i-1} p_k p_j$

$s(i, j) \leftarrow k$

repeat

repeat

end Matrix-Chain-Order

算法实例



$n=6$

矩阵
规模

A_1
 30×35

A_2
 35×15

A_3
 15×5

A_4
 5×10

A_5
 10×20

A_6
 20×25

$m(i,j)$	$j=1$	2	3	4	5	6
$i=1$	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

$s(i,j)$	$j=1$	2	3	4	5	6
$i=1$	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5

$m(1,2) > m(1,3) ?$

$30 \times 35 \times 15 > 30 \times 35 \times 5 + 35 \times 15 \times 5$

$(A_1 (A_2 A_3))((A_4 A_5) A_6)$

6.8 流水线调度问题

- 问题描述
- 两种调度方式
- 调度 S 完成的时间
- 递推关系式分析
- 递推关系式推演
- 调度规则
- 应用实例



问题描述

- 处理一个作业通常需要若干个不同的任务。
- 假设有 n 个作业，每个作业 i 要求执行 m 个任务： $T_{1i}, T_{2i}, \dots, T_{mi}, 1 \leq i \leq n$ 。
 - 任务 T_{ji} 只能在设备 P_j 上执行， $1 \leq j \leq m$ 。
 - 对于任一作业 i ，在任务 $T_{j-1,i}$ 没完成以前，不能对任务 T_{ji} 开始处理。
 - 同一台设备在任何时刻不能同时处理一个以上的任务。

作业1: $T_{11} \quad T_{21} \quad T_{31} \quad \dots \quad T_{m1}$
 作业2: $T_{12} \quad T_{22} \quad T_{32} \quad \dots \quad T_{m2}$
 ...

设备 P_3

问题描述

- 假设完成任务 T_{ji} 所要求的时间是 t_{ji} , $1 \leq j \leq m$, $1 \leq i \leq n$, 那么如何将这 $n \times m$ 个任务分配给这 m 台设备, 才能使这 n 个作业在以上要求下顺利完成呢?

要确定每台设备上: 1).任务的执行顺序; 2).每个任务的开始执行时间

调度

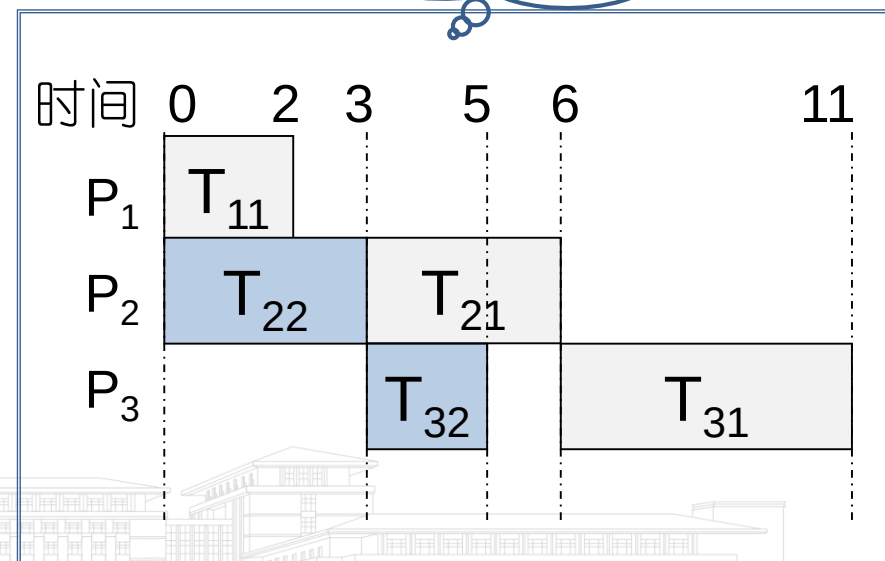
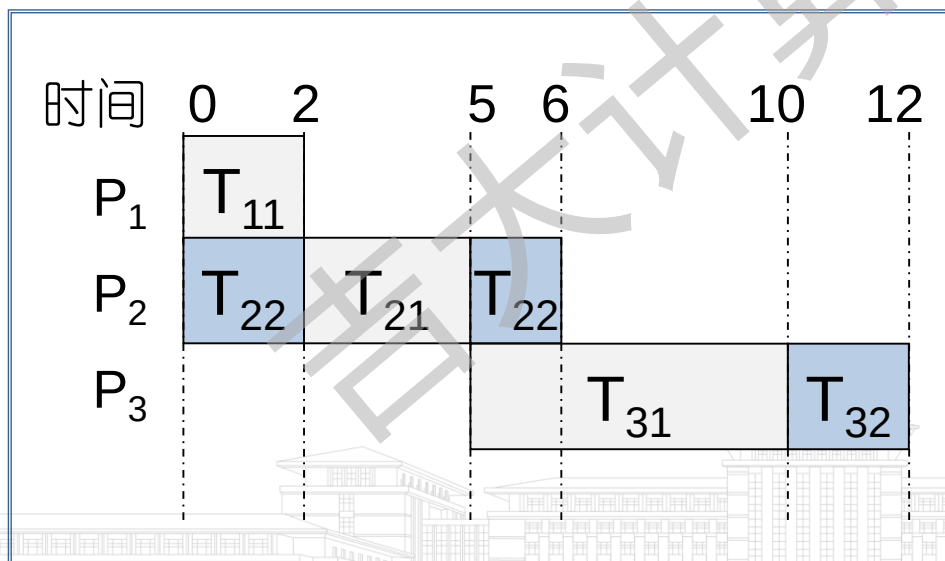
作业1: T_{11} T_{21} T_{31} ... T_{m1}
 作业2: T_{12} T_{22} T_{32} ... T_{m2}

设备 P_3

两种调度方式

- 非抢先调度：当前任务一旦被设备处理就不能被中断，直到完成。
- 抢先调度：无上述要求。
- 一个例子：考虑在3台设备上调度两个作业，每个作业包含三个任务，任务要求的时间矩阵J如下所示，那么有两种可能的调度：

$$J = \begin{pmatrix} 2 & 0 \\ 3 & 3 \\ 5 & 2 \end{pmatrix}$$



非抢先调度

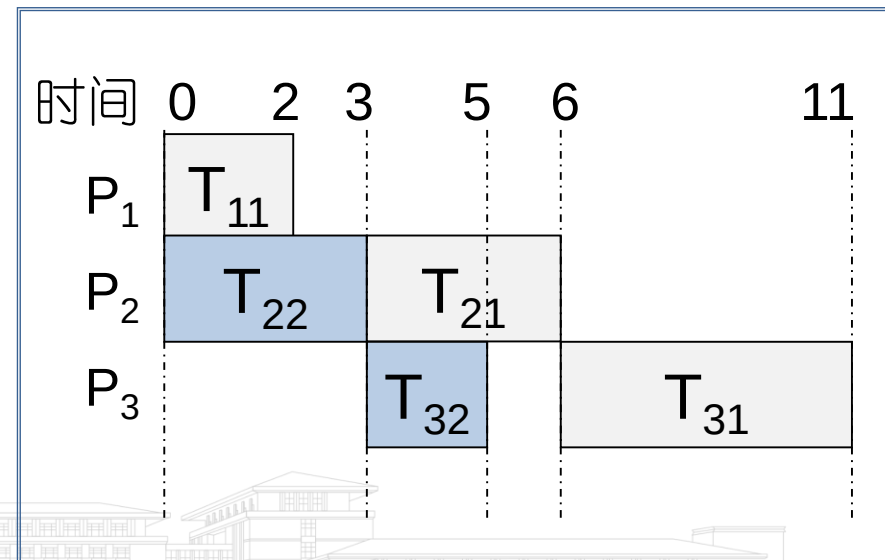
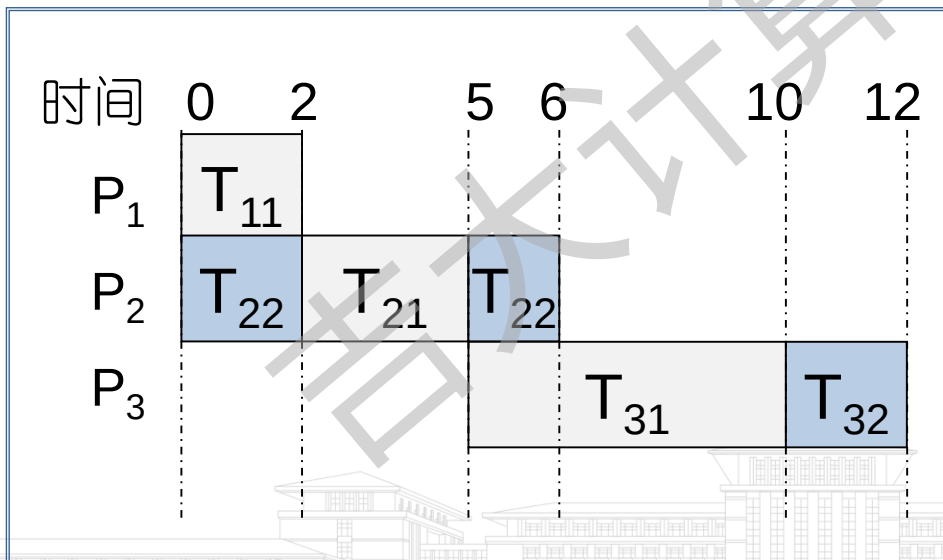
调度S完成的时间

- 作业i的完成时间 $f_i(S)$ ：在调度方案S下作业i的所有任务得以完成的时间。
- 调度S完成的时间 $F(S)=\max\{f_i(S)\}$, $1 \leq i \leq n$ 。

$$f_1(S)=10; f_2(S)=12$$

$$f_1(S)=11; f_2(S)=5$$

$$J = \begin{pmatrix} 2 & 0 \\ 3 & 3 \\ 5 & 2 \end{pmatrix}$$



非抢先调度下最优时间OFT

OFT：非抢先调度下最优时间； POFT：抢先调度下最优时间。

$m > 2$ 时，求OFT调度和POFT调度问题都是难于计算的问题。这里只讨论 $m = 2$ 时，获取OFT调度这一特殊情况。

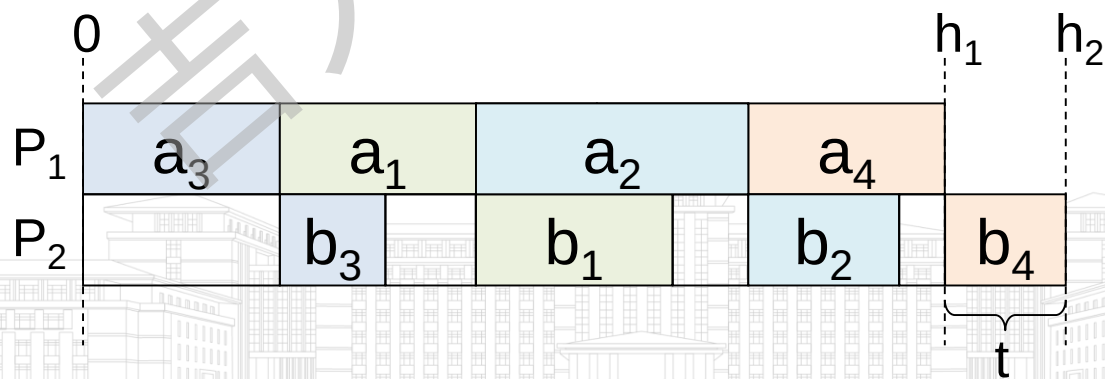
- 在 $m = 2$ 的OFT调度问题中，每个作业只有两个任务。为简便起见，令 a_i 表示 t_{1i} , b_i 表示 t_{2i} ，假定 $a_i \neq 0$ ， $1 \leq i \leq n$ 。
- 每个任务都应在最早的可能时间开始执行，排列在后面的作业可以先被执行。
- 若存在 $a_i = 0$ 的作业，那么首先对于所有 $a_i \neq 0$ 的作业求出一种最优调度的排列，然后把所有 $a_i = 0$ 的作业以任意次序加到这一排列的前面。



递推关系式分析

最优调度的排列具有下述性质：在给出了这个排列的第一个作业后，剩下的排列相对于这两台设备在完成第一个作业时所处的状态而言是最优的。

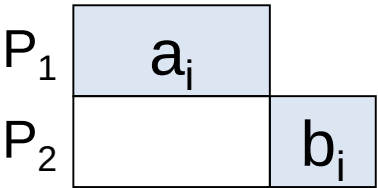
- 令 t 表示设备 P_1 完成任务后，设备 P_2 还需要花费的执行时间，即后继任务在 P_1 可用后，还需要等待 t 时间才可以使用 P_2 。
- 设 $g(S, t)$ 是上述 t 下作业集合 S 的最优调度长度。
- 作业集合 $\{1, 2, \dots, n\}$ 的最优长度是 $g(\{1, 2, \dots, n\}, 0)$ 。即作业集合 $\{1, 2, \dots, n\}$ 需要在 P_1 可用后，再等待 0 时间就可以使用 P_2 。





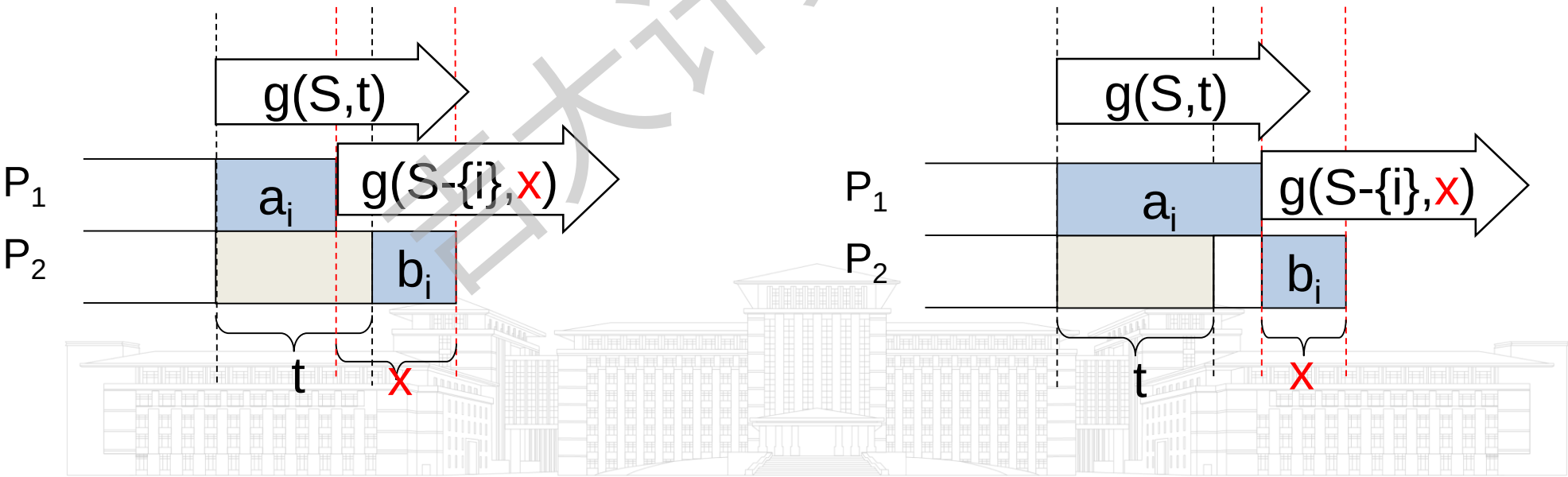
递推关系式分析

对于原问题, $g(\{1,2,\dots,n\},0) = \min_{1 \leq i \leq n} \{a_i + g(\{1,2,\dots,n\} - \{i\}, b_i)\}$



递推关系式: $g(S,t) = \min_{i \in S} \{a_i + g(S - \{i\}, x)\}$ $x = b_i + \max\{t - a_i, 0\}$

初始值: $g(\emptyset, t) = \max\{t, 0\}$





递推关系式推演

$$g(S,t) = \min_{i \in S} \{a_i + g(S - \{i\}, b_i + \max\{t - a_i, 0\})\}$$

初始值: $g(\emptyset, t) = \max\{t, 0\}$

- 设 i 和 j 是 S 的一个最优调度 R 中排在前面的两个作业

- $g(S, t)$

$$= a_i + g(S - \{i\}, b_i + \max\{t - a_i, 0\})$$

$$= a_i + a_j + g(S - \{i, j\}, \underbrace{b_j + \max\{b_i + \max\{t - a_i, 0\} - a_j, 0\}}_{t_{ij}})$$

$$= a_i + a_j + g(S - \{i, j\}, t_{ij})$$

- 将作业 i, j 在 R 中易位, 获得 $g'(S, t) = a_i + a_j + g(S - \{i, j\}, t_{ji})$

$$t_{ij} \leq t_{ji}$$

思考: 若要 $g(S, t) \leq g'(S, t)$, t_{ij} 与 t_{ji} 应满足怎样的不等式关系?

递推关系式推演

分离出 $b_i - a_j$

$$t_{ij} = b_j + \max\{b_i + \max\{t - a_i, 0\} - a_j, 0\}$$

$$= b_j + b_i - a_j + \max\{\max\{t - a_i, 0\}, a_j - b_i\}$$

分离出 $-a_i$

$$= b_j + b_i - a_j + \max\{t - a_i, a_j - b_i, 0\}$$

$$= b_j + b_i - a_j - a_i + \max\{t, a_j + a_i - b_i, a_i\}$$

$$t_{ji} = b_j + b_i - a_j - a_i + \max\{t, a_j + a_i - b_j, a_j\}$$

由上述推导可知,若要 $g(S,t) \leq g'(S,t)$, 则 $t_{ij} \leq t_{ji}$, 必有:

$\max\{t, a_j + a_i - b_i, a_i\} \leq \max\{t, a_j + a_i - b_j, a_j\}$, 即考虑:

$\max\{a_j + a_i - b_i, a_i\} \leq \max\{a_i + a_j - b_j, a_j\}$, 不等式两侧同时减去 a_j 和 a_i ,
得到 $\max\{-b_i, -a_j\} \leq \max\{-b_j, -a_i\}$, 简化为 $\min\{b_i, a_j\} \geq \min\{b_j, a_i\}$



调度规则设计

怎样设计调度能满足min不等式关系？

结论： $\min\{b_i, a_j\} \geq \min\{b_j, a_i\} \longrightarrow t_{ij} \leq t_{ji} \longrightarrow g(S, t) \leq g'(S, t)$

- 对于作业i和j，当 a_i 或 b_j 最小时，min不等式成立，即先i后j处理作业长度更短。
- 若一个作业调度中，每一对相邻的作业都满足min不等式关系，则获得一个最优长度，该调度也是一个最优调度。
- 推知
 - 如果 $\min\{a_1, a_2, \dots, a_n, b_1, b_2, \dots, b_n\} = a_i$ ，那么作业i是最优调度中的第一个作业。
 - 如果 $\min\{a_1, a_2, \dots, a_n, b_1, b_2, \dots, b_n\} = b_j$ ，那么作业j是最优调度中的最后一个作业。



最优调度规则

- 把全部 a_i 和 b_j 非降次序排序后，考察该序列：
 - 如果下一个元素是 a_j 且作业 j 还没调度，那么在还没使用的最左位置调度作业 j ；
 - 如果下一个元素是 b_j 且作业 j 还没调度，那么在还没使用的最右位置调度作业 j ；
 - 如果已经调度了作业 j ，则考查下一个元素，直到 n 个作业都分配完为止。

该规则也适用于存在 $a_i=0$ 的作业集合。



应用实例



- 设 $n=4$, $(a_1, a_2, a_3, a_4)=(3, 4, 8, 10)$, $(b_1, b_2, b_3, b_4)=(6, 2, 9, 15)$

(1) 按非降次序排列, 获得排序:

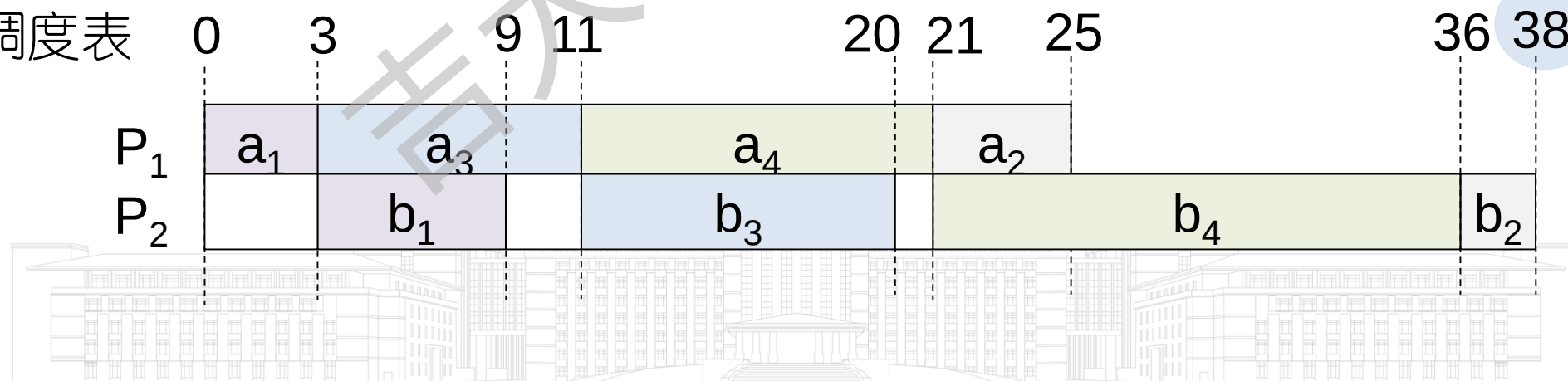
b_2	a_1	a_2	b_1	a_3	b_3	a_4	b_4
2	3	4	6	8	9	10	15

(2) 调度序列

b_1	b_2	b_3	b_4
1	3	4	2

最优解

(3) 调度表



6.9 小结

- 动态规划适用的问题特点
 - 多阶段决策优化问题
- 动态规划方法特点
 - 最优子结构性质
 - 重叠子问题性质
- 动态规划方法的时间复杂度受存储空间的影响
 - 备忘录：记录子问题最优解
 - 标记函数：记录对应最优解的决策序列





6.9 小结

- 动态规划和分治法的区别
 - 分治法不具有重叠子问题性质
- 动态规划和贪心法的区别
 - 动态规划在做下一步决策时，依赖子问题的最优解
 - 贪心法在做下一步决策时，不考虑子问题结果，只顾眼前



6.9 小结

- 6.1 一般方法
 - 掌握最优性原理、动态规划适用的问题特点、求解思想、动态规划的特点等基本知识。能掌握动态规划求解问题的一般过程。
- 6.2 多段图
- 6.3 货郎担问题
 - 能掌握最优性原理证明和递推关系式的一般设计方法，动态规划算法的一般设计思想和时间复杂度分析方法。



6.9 小结

- 6.4 0/1背包问题
- 6.5 可靠性设计
 - 基于动态规划递推关系式演变算法，掌握序偶对方法，深入理解存储空间对时间复杂度的影响。
- 6.6 最优二分检索树
- 6.7 矩阵链乘积问题
 - 掌握复杂问题的递推关系式设计方法，掌握优化算法和降低复杂度的设计技巧
- 6.8 流水线调度问题
 - 掌握调度规则，理解递推关系式数学推演为算法带来的简化作用。



6.9 小结

- 总结优化技巧：
 - 0/1背包问题序偶对法：抽取递推关系式中关键信息、并利用贪心法和估计函数降低空间复杂度
 - 最优二分检索树：减少子问题范围
 - 流水线调度问题：从递推关系式入手，经过数学推演获得一个易于执行的规则

能够识别出适合动态规划的可计算性问题、独立设计算法和分析算法复杂度。





本章结束

